

OpenFrame OSC管理者ガイド

OpenFrame/OSC 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame OSC管理者ガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/OSC 7.1

ガイドバージョン: v3.1.2

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの`$(INSTALL_PATH)/license/oss_licenses`に記載されている事項を参照してください。

目次

このガイドについて	ix
第1章 概要	1
1.1. OpenFrame/OSCの特徴	1
1.2. OSCシステムの構造	3
1.2.1. OSCシステム・サーバー	4
1.2.2. OSCユーザー・サーバー	5
1.2.3. OSCリージョン	5
1.2.4. OSCリージョンの接続	7
1.2.5. OpenFrame Gateway (OpenFrame GW)	7
1.3. OSCの起動と終了	8
1.3.1. OSCの起動	8
1.3.2. OSCの終了	8
第2章 OSCシステムの設定	11
2.1. OSCの設定	11
2.1.1. ofsys.seq	12
2.1.2. ofosc.seq	12
2.1.3. osc.region.list	12
2.2. Tmax環境設定ファイルの設定	13
2.2.1. サーバー・グループの設定	13
2.2.2. システム・サーバーの設定	14
2.2.3. システム・サーバー・サービスの設定	14
2.2.4. ユーザー・サーバーの設定	15
2.3. TCacheの設定	15
2.4. WASサーブレットの設定	16
2.4.1. oschttp	16
第3章 OSCリージョンの設定	19
3.1. 概要	19
3.2. アプリケーション・サーバー・バイナリの作成	19
3.2.1. ソース・ファイルの作成	19
3.2.2. コンパイルとデプロイ	21
3.3. アプリケーション・サーバーDBテーブルの作成	21
3.4. アプリケーション・サーバー・データセットの作成	24
3.4.1. パーティション内TDQデータセットの作成	25
3.4.2. TSQデータセットの作成	26
3.4.3. FILEデータセットの作成	27
3.5. アプリケーション・サーバー・リソースの登録	28
3.5.1. デフォルト・リソースの登録	28
3.5.2. ユーザー・リソースの登録	29
3.6. Tmax環境設定ファイルの設定	29

3.6.1.	アプリケーション・サーバーとサービスの登録	29
3.6.2.	アプリケーション補助サーバーとサービスの登録	31
第4章	OSCアプリケーション	33
4.1.	概要	33
4.2.	ソースの前処理	33
4.2.1.	cobolprep	34
4.2.2.	osccobprep	38
4.2.3.	osccbldpp	36
4.2.4.	osclipp	37
4.2.5.	osccprep	38
4.3.	コンパイル	39
4.4.	デブロイおよび適用	39
4.5.	プログラムとトランザクションの登録	41
第5章	OSCの運用	43
5.1.	OSCの起動と終了	43
5.1.1.	OSCの起動	43
5.1.2.	OSCの終了	44
5.2.	OSCの運用	46
5.2.1.	システムの運用	46
5.2.2.	リージョンの運用	47
5.2.3.	ログ・レベルの管理	49
5.3.	セキュリティ管理	50
5.3.1.	ユーザー・セキュリティ	50
5.3.2.	リソース・セキュリティ	51
5.4.	ログ情報	53
5.4.1.	Tmaxログ	53
5.4.2.	サーバー・ログ	53
5.4.3.	サービス・ログ	55
5.4.4.	操作ログ	56
5.4.5.	監視ログ	56
5.5.	サーバーの復旧	57
5.5.1.	OSCシステム・サーバーの復旧	57
5.5.2.	OSCアプリケーション・サーバーと補助サーバーの復旧	58
5.6.	日時の設定	58
5.6.1.	日時設定の準備	59
5.6.2.	日時の設定	59
5.6.3.	日時の変更	60
5.7.	リソースの自動インストール	61
5.7.1.	プログラムの自動インストール	61
付録 A.	Tmax環境設定ファイルの例	65
付録 B.	TCache設定ファイルの例	69

索引	71
----------	----

図目次

[図 1.1] OSCシステムの構造	4
[図 5.1] デフォルト・トランザクションCESN画面	51

このガイドについて

対象読者

Tmax OpenFrame[®] (以下OpenFrame)は、IBMメインフレーム・システムをオープン・システム環境であるUNIXに移行するリHOST・ソリューションです。

OpenFrame/OSCは、IBMメインフレーム・システムで行われていたトランザクション処理を担当するシステムです。OpenFrame/OSCは、安定性と優れた性能を持つTPモニターであるTmaxSoftのTmaxエンジンをベースとしています。

本書は、OpenFrame Onlineを構成するシステムのうち、IBMメインフレームのIBM CICSトランザクション・サーバー(以下CICS)に対応するOnline System type C(以下OSC)を運用・管理するユーザーを対象としています。

参考

OSCシステムの全般的な概念については「[第1章 概要](#)」を参照してください。

前提知識

OSCシステムの概念を理解するには、以下の事項を熟知している必要があります。

- UNIXシステム
- TmaxSoftのTPモニターであるTmax
- IBM CICSトランザクション・サーバー・システム
- OpenFrame Managerの使用方法

制限事項

本書では、OpenFrame/OSCの運用方法と運用に必要な設定について説明しています。OpenFrameの基盤環境であるUNIX、OpenFrameのエンジンであるTmax、そしてOSCシステムへのリHOSTの対象であるメインフレームCICSについては、該当するガイドを参照してください。

本書の構成

本書は、計5つの章と付録で構成されています。

各章の主要内容は以下のとおりです。

- 第1章: 概要

OpenFrame/OSCシステムの概念とシステム構造について説明します。

- 第2章: OSCシステムの設定

OSCシステムの起動に必要な環境構成について説明します。

- 第3章: OSCリージョンの設定

OSCリージョンの構成に必要な設定および準備プロセスについて説明します。

- 第4章: OSCアプリケーション

アプリケーション・ロジックが実装されるプログラムを実行する前に行うべき作業について説明します。

- 第5章: OSCの運用

起動・終了プロセス、セキュリティ、ログ記録方法など、OSCの運用および管理方法について説明します。

- 付録A : Tmax環境設定ファイルの例

Tmax環境設定ファイルの例を紹介します。

- 付録B : TCache設定ファイルの例

TCache設定ファイルの例を紹介します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名、ディレクトリ
<Ctrl>+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上(32bit、64bit)
	Linux x86 2.6以上(32bit、64bit)
	Linux ia64 2.6以上(32bit、64bit)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 FS07
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、TmaxSoftの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame OSCインストールガイド	OpenFrame製品のインストールと環境設定について記述しています。
OpenFrame OSC開発者ガイド	OSCシステムで提供する開発リソース情報を含む開発者のための内容を記述しています。
OpenFrame OSCリソース定義ガイド	OSCシステムを使用するために必要なリソース定義について記述しています。
OpenFrame OSCマッピングサポートガイド	TN3270端末との通信インターフェースであるマッピング・サポートについて記述しています。
OpenFrame エラーメッセージ リファレンスガイド	製品の使用時に発生し得るエラーに関する情報とその対応策について記述しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用に使用する多様なツール・プログラムについて記述しています。
OpenFrame マイグレーションガイド	メインフレーム環境のリソースをOpenFrame環境に移行するときに必要な情報や移行プロセス、注意事項などについて記述しています。
OpenFrame Manager ユーザーガイド	OpenFrame Managerの各メニュー画面の概要と使用方法について記述しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

参考文献

製品	ガイド
メインフレーム	CICS System Programming Reference Guide
	CICS Customization Guide

第1章 概要

本章では、OpenFrame/OSCシステムの特徴と構造、および起動方法について説明します。

1.1. OpenFrame/OSCの特徴

OpenFrame/OSC(以下、OSC)は、メインフレーム・リホスト・ソリューションのOpenFrameを構成する製品の1つであり、CICSのオンライン業務を簡単な移行手続きでオープン・システムで運用できるようにする製品です。

一般的に、メインフレームのオンライン業務システムをオープン・システムに移行するとき、以下のような課題が想定されます。

- メインフレームの機能、性能および安定性をどのように提供するか
- 簡単に移行できるか

OSCは、メインフレームCICSと同等の機能を提供します。性能と安定性の問題を解決するために、オープン・システムでの性能と安定性が検証されたTPモニターのTmaxエンジンをベースとしています。

TmaxエンジンをベースにしたOSCは、次のようなTmaxの特徴を持ちます。

- 便利なプロセス管理

OSCで作成されたプロセスはTmaxにより起動から終了まで管理されます。Tmaxが提供する多様なモニタリング情報を使用できるので、プロセスの管理が容易になります。

- 大容量トランザクションのサポート

Tmaxには、大容量トランザクション処理のスケジューリング機能やサービス・キューの管理機能が組み込まれています。Tmaxはオープン環境のミッション・クリティカルな業務システムに適合しています。TmaxをベースにするOSCも大容量のトランザクションを安定的にサポートします。

- ロード・バランシングとフェイルオーバーのサポート

IBMメインフレームのマシンに比べ、オープン・システムのマシンは小型です。そのため、IBMメインフレームの業務をオープン・システムに移行する場合は、2台以上のマシンでハードウェアを構成します。2台以上のハードウェアでアプリケーションを構成するには、マシン間の負荷分散や障害対策が重要になります。OSCでは、Tmaxのロード・バランシングおよびフェイルオーバー機能により、ハードウェア・レベルの問題が解決されます。

- オープン環境における自由な連携

連携はリホストに直接関わる問題ではありません。しかし、システムを拡張したり他のシステムと連携したりする状況は頻繁に発生します。TmaxはX/Open DTPモデルに準拠するTuxedoのよう

な他の商用TPモニターと連携することができ、OSCで作成されたアプリケーション・システムは容易に拡張できるので、OSCは他のリホスト・ソリューションに比べて優れたメリットがあるといえます。さらに、弊社のWebアプリケーション・サーバーであるJEUSなどのWeb環境との連携も可能です。TmaxをベースにしたOSCは、大量高速処理が必要な大型オンライン・アプリケーション・システムのリホストを実現します。

OSCは、CICSのトランザクション・サービスをUNIXなどのオープン環境でも同じレベルで提供します。さらに、CICSの各種アプリケーション・プログラムを、業務ロジックやコードの修正なしにオープン環境でも同様に運用できるようにします。OSCはマイグレーションを容易にするために、IBMメインフレームCICSの業務をそのまま適用できるインターフェースを提供しています。また、CICS上で開発されたユーザー・プログラムで使用されるEXEC CICS文、EXEC DLI文、およびCBLTDLI関数も提供しています。

OSCシステムのその他の特徴は、以下のとおりです。

- **各サーバー・プロセスは最適な環境下で業務処理**

- OSCアプリケーション・サーバーは1つ以上のプロセスで構成されます。クライアントから多数のトランザクション要求があった場合、プロセス・スケジューリング機能を使用して最適な業務環境でトランザクション要求を処理します。
- 業務負荷に合わせて自動でサーバー・プロセス数を制御し、効率的な分散処理を行います。

- **トランザクションの並列処理が可能**

- 設定ファイルやRTSD(Run-Time System Definition)などの機能により、同じアプリケーション・サーバーの全プロセスが同時に同じリソース・サービスを提供することができます。
- 1つのアプリケーション・サーバーと補助サーバーは、メインフレームCICSの1つのリージョンにマッピングされます。OSCアプリケーション・サーバーが2つ以上のプロセスで構成されている場合も、内部で同一の設定、リソース設定およびリソース定義を管理します。

- **CICSアプリケーション・プログラムを業務ロジックの変更なしに、そのままOSCアプリケーション・サーバーに適用して動作可能**

- OSCは、メインフレームのサブシステムに対応するOpenFrame製品を利用して、CICSシステムのリソース・サービスと同等のサービスを提供します。
- CICSが提供するリソースを変更せずに移行できるため、既存のユーザー・アプリケーションを簡単なマイグレーション作業によりオープン・システムで使用できます。

以下は、OSCシステムが提供するサービスの一覧です。

- Abend機能
- TACFによる認証/承認
- チャンネル・コマンド
- 環境サービス
- 例外

- TSAMによるファイル制御
 - インターバル制御
 - ジャーナリング
 - マッピング・サポート
 - 名前付きカウンター・サーバー
 - プログラム制御
 - セキュリティ
 - TJESによるスプール・インターフェース
 - 同期点 (Syncpoint)
 - ストレージ制御
 - タスク制御
 - 一時ストレージ制御
 - 端末制御
 - 一時データ制御
- **便利なシステム管理ツールおよびユーティリティを提供**
 - SysMaster製品と連携し、サーバー・プロセスの状態、サービス数、サービスの処理時間などをモニタリングできます。
 - TPモニター・システム管理ユーティリティのtmadminと一緒に活用できます。

参考

tmadminは、弊社のTPモニター製品Tmaxのユーティリティです。

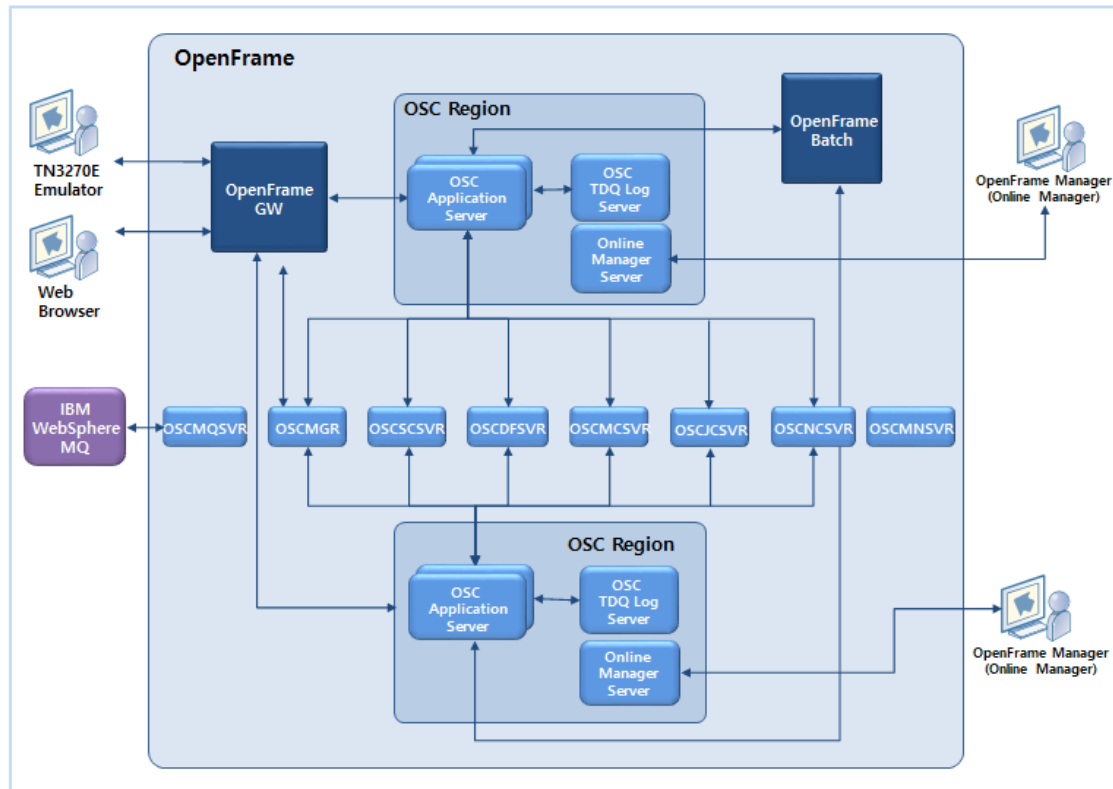
- **GUIのOpenFrame Managerの[OSC]メニューを提供してユーザー利便性を向上**
 - OSCシステム・ログの表示、サーバーのリソース定義の作成、表示、変更を実行できます。
 - OSCサーバーの状態、サーバー情報、および動的な端末情報など、システムの動的な状態表示サービス機能を提供します。
 - メインフレームCICS提供のトランザクションに対応する機能をGUIで提供します。

1.2. OSCシステムの構造

以下は、OSCシステムの構造を示した図です。OSCシステムは、OSCシステム全体の運用のためのOSCシステム・サーバー、OSCリージョンを構成するアプリケーション・サーバーと補助サーバー、

およびOpenFrame Gatewayで構成されます。OSCアプリケーション・サーバーと補助サーバーは、CICSの1つのリージョンに対応します。

[図 1.1] OSCシステムの構造



1.2.1. OSCシステム・サーバー

以下は、OSCシステム・サーバーの一覧です。

システム・サーバー	説明
OSCMGR	OSCの全リージョンの管理を担当するサーバーです。リージョンの起動と終了、SPI、TX_TIMEの設定機能を提供します。
OSCSCSVR	時間条件が設定されているトランザクションをスケジューリングするサーバーです。アプリケーション・サーバーと内部プロトコルを使用して通信します。
OSCDFSVR	OSCアプリケーション・サーバーのEDF機能を管理するサーバーです。OSCアプリケーション・サーバーやOpenFrame Managerと通信します。
OSCMCSVR	OSCアプリケーション・サーバーをモニタリングするサーバーです。
OSCNCSVR	名前付きカウンターを処理するサーバーで、内部的にUNIXファイル・システムを使用します。OSCアプリケーション・サーバーと内部プロトコルを使用して通信します。

システム・サーバー	説明
OSCJCSVR	OSC JOURNALMODEL リソースのTYPE MVSをサポートするためのサーバーです。
OSCMNSVR	OSCシステムのサーバー・プロセスをモニタリングするための追加サーバーです。サーバー・プロセスをモニタリングしない場合は、設定しなくてもシステムの動作に影響はありません。

1.2.2. OSCユーザー・サーバー

以下は、OSCが提供するユーザー・サーバーです。該当する機能が必要な場合に、ユーザーが追加するサーバーです。

ユーザー・サーバー	説明
OSCMQSVR	IBM Websphere MQ製品と連携して、OSCトランザクションのためのキュー・トリガー・モニター機能を提供するユーザー・サーバーです。キュー・トリガー・モニター機能を使用しない場合、設定しなくてもシステムの動作に影響はありません。

1.2.3. OSCリージョン

OSCリージョンは、メインフレームのCICSリージョンに対応する独立した領域です。各リージョンは、互いに異なる設定とリソース定義をベースにユーザー・トランザクション・サービスを提供します。リージョンごとに設定ファイルが存在し、リソース定義はOSC SDテーブルを使用して管理します。

OSCシステムで1つのリージョンは、1つ以上のアプリケーション・サーバー・プロセスと補助サーバーで構成されます。

OSCアプリケーション・サーバーを多数のサーバー・プロセスで設定する場合、各プロセスは同一プログラムのコピーを動的にロードして同時に同じまたは異なるトランザクション・サービスを提供します。OSCアプリケーション・サーバーはマルチタスキングが可能です。多数のサーバー・プロセスが同じサービスを提供するためには、同じリソース定義を使用する必要があります。全プロセスは同一の設定とリソース定義を参照します。エンジンとサーバーの設定ファイルはエンジンの起動時にシステム内部で動的コピーが作られるので、全サーバー・プロセスは起動時間に関係なく同じ設定でサービスを提供します。リソース定義の参照は、指定のRTSD(Runtime System Definition) データベース・テーブルにリソース定義を保存し、それにアクセスすることで実現します。

補助サーバー

OSCリージョンは、トランザクション・サービスを担当するアプリケーション・サーバー・プロセスの外に、多様な機能をサポートする補助サーバーを提供します。

補助サーバー	説明
OSCTLSVR	ログ・タイプのTDQを処理するOSC TDQログ・サーバーです。内部的にUNIXファイル・システムにアクセスし、パフォーマンスを向上させるために内部バッファリングを行います。OSCアプリケーション・サーバーとTCP/IPプロトコルを使用して通信します。 アプリケーション・サーバーがログ・タイプのTDQを使用しない場合は設定する必要がありません。
OSCOSSVR	OpenFrame Managerの多様なサービスを提供するためのサーバーです。OpenFrame Managerを使用する場合は、必ず設定する必要があります。

リソース・マネージャー

アプリケーション・サーバー・プロセスは、アプリケーション・プログラムに多様なサービスを提供するために各種リソースを提供します。リソースはリソース・マネージャーによって管理されます。

リソース・マネージャーはサポートする機能の特性によって以下のように区分できます。

● サービス管理リソース・マネージャー

OSCサーバーの多様なサービスを管理します。

リソース・マネージャー名	説明
Program Control	プログラムの実行を担当します。
Task Control	タスク間の接続を担当します。
Interval Control	時間関連の操作を担当します。
Storage	メモリの動的割り当てと返還処理を担当します。
TACF	セキュリティを担当します。
TSQ	一時保存キューを管理します。
TDQ	データ送信キューを管理します。
Journal Control	ジャーナル管理を担当します。

● データ管理リソース・マネージャー

データの保存および表示などを管理します。

リソース・マネージャー名	説明
File Control	TSAMデータセットのアクセスを管理します。
DL/I control	IMS/DBのアクセスを管理します。
NCS	名前付きカウンター・サーバーのアクセスを管理します。

- データ通信リソース・マネージャー

端末や他のシステムとの通信を管理します。

リソース・マネージャー名	説明
Terminal	端末の接続と管理を担当します。
Mapping Support	マップの変換処理を担当します。
Spool	TJESシステムのインターフェースを担当します。

リソースを管理するリソース・マネージャーは共有ライブラリの形でユーザーに提供されます。リソース・マネージャーは、アプリケーション・サーバー・プロセスのバイナリが作成される時に動的にリンクされます。アプリケーション・サーバーの作成時に自動でリンクされるため、ユーザーは関与する必要がありません。ユーザーはEXEC CICSまたはEXEC DLIコマンドを使用して、アプリケーション・プログラムでリソースを利用できます。

参考

各リソースおよびリソース・マネージャーについては、『OpenFrame OSC開発者ガイド』を参照してください。

1.2.4. OSCリージョンの接続

TN3270/TN3270EエミュレーターとEXCIクライアントにより、OSCリージョンに接続できます。

区分	説明
TN3270/TN3270E エミュレーター	パソコンでIBM 3270端末をエミュレーションするプログラムです。このエミュレーターを利用してOSCリージョンに接続し、アプリケーションを実行できます。 まずOpenFrame Gatewayに接続した後、LOGON APPLID（リージョン名）コマンドを入力して特定のOSCリージョンに接続します。
EXCIクライアント	OpenFrame/BatchプログラムからOSCシステムにアクセスする際に利用します。EXCIプログラミング・インターフェースを使用します。

1.2.5. OpenFrame Gateway (OpenFrame GW)

TN3270/TN3270EエミュレーターやWebブラウザーを介して接続する端末を管理し、OSCアプリケーション・サーバーにトランザクションを要求するサーバーです。

参考

OpenFrame Gatewayの詳細については、『OpenFrame GW 管理者ガイド』を参照してください。

1.3. OSCの起動と終了

OSCの起動と終了は、OpenFrameシステム・サーバーの起動/終了段階とOSCシステム・サーバーおよびOSCリージョンの起動/終了段階に分けられます。各段階のより詳しい説明は、[「5.1. OSCの起動と終了」](#)を参照してください。

1.3.1. OSCの起動

OSCの起動は、OpenFrameシステム・サーバーを起動する段階、OSCシステム・サーバーおよびユーザー・サーバーを起動する段階、最後にOSCリージョンを起動する段階に分けられます。

1. OpenFrameシステム・サーバーの起動

OpenFrameシステム・サーバーを設定ファイル(ofsys.seq)に従って順次に起動します。

2. OSCシステム・サーバーおよびOSCユーザー・サーバーの起動

OSCシステム・サーバーを起動し、設定ファイル(ofosc.seq)に従ってOSCユーザー・サーバーを順次に起動します。

3. OSCリージョンの起動

まずOSCリージョンのための共有メモリ、ディレクトリ、ファイルなどのリソースを作成した後、OSCリージョン・サーバーを起動します。OSCリージョンの起動時に、PLT(Program List Table)に登録されている後処理プログラムが実行されます。初期化プログラムの実行が終了すると、OSCリージョンの起動が完了します。

1.3.2. OSCの終了

OSCの終了は、OSCのリージョンを終了する段階、OSCユーザー・サーバーおよびOSCシステム・サーバーを終了する段階、OpenFrameシステム・サーバーを終了する段階に分けられます。

1. OSCリージョンの終了

終了するOSCリージョンのPLT(Program List Table)に登録されている前処理プログラムを実行します。OSCリージョン・サーバーを終了後、OSCリージョンが使用した共有メモリ、ディレクトリ、ファイルなどのリソースを削除し、OSCリージョンを終了します。

2. OSCユーザー・サーバーおよびOSCシステム・サーバーの終了

設定ファイル(ofosc.seq)に従って、起動時と逆の順序でOSCユーザー・サーバーを終了し、OSCシステム・サーバーを終了します。

3. OpenFrameシステム・サーバーの終了

設定ファイル(ofsys.seq)に従って、OpenFrameシステム・サーバーを起動時と逆の順序で終了します。

第2章 OSCシステムの設定

OSCリージョンを運用するには、OpenFrame/OSCの設定とOpenFrameのベース環境を構成するTmaxの環境設定ファイルの設定が必要です。本章では、OSCシステムの起動に必要な環境構成について説明します。

2.1. OSCの設定

OSCシステムを起動するには、OpenFrameシステム情報とアプリケーション・サーバー、OSC TDQ ログ・サーバーの情報と運用に関する情報が設定されている必要があります。

- OSC設定ファイル

以下は、OSC設定ファイルの説明です。詳細については、各節の説明を参照してください。

設定ファイル	説明
ofsys.seq	OpenFrame/Base、OpenFrame/BatchなどのOpenFrameシステム・サーバー名をそれぞれ改行して記述します。このファイルは、システムの起動と終了時に使用されます。
ofosc.seq	OSCMQSVRなどのOSCユーザー・サーバーを使用する場合、このファイルにそれぞれのサーバー名を改行して記述します。このファイルは、システムの起動と終了時に使用されます。
osc.region.list	OSCリージョン名をそれぞれ改行して記述します。このファイルは、システムの起動と終了時に使用されます。

- OSC環境設定

OSCの運用に関するシステム設定を、openframe_osc.confの各セクションのキー値に設定した後、**ofconfig**ツールを使用して保存します。

以下は、各サブジェクトの説明です。

サブジェクト	説明
cobnet	.NETアプリケーションでCOBOLアプリケーションを呼び出すために提供するcobnetサーバーで使用する設定を行います。
osc	OSCシステムの共通情報を設定します。
osc.{servername}	OSCアプリケーション・サーバーで使用する設定を行います。
osc.{osctlsvrname}	OSCリージョンでログ・タイプTDQを使用する場合、TDQログ・サーバーで使用する設定を行います。

サブジェクト	説明
osc.{oscmqsvrname}	IBM WebSphere MQのQueue Trigger Monitorを利用するために提供するサーバーで使用する設定を行います。

参考

サブジェクトの設定方法については、『OpenFrame 環境設定ガイド』を参照してください。

2.1.1. ofsys.seq

ofsys.seqは、OpenFrameシステム・サーバーを設定するファイルであり、OSCシステムの起動と終了時に使用されます。OpenFrameシステム・サーバー名を起動する順に改行して記述します。システムの終了時には、起動時と逆の順序で終了します。

以下は、ofsys.seqファイルにofrlmsvr、ofrlmwrk、ofrsmlogの3つのOpenFrameシステム・サーバーを設定した例です。

```
ofrlmsvr
ofrlmwrk
ofrsmlog
```

2.1.2. ofosc.seq

ofosc.seqは、OSCシステムの起動時に起動するOSCユーザー・サーバーを設定するファイルであり、OSCシステムの起動と終了時に使用されます。ユーザー・サーバー名を起動する順に改行して記述します。OSCMQSVRを使用する場合に設定します。

以下は、ofosc.seqファイルにOSCMQ001というOSCユーザー・サーバーを設定した例です。

```
OSCMQ001
```

2.1.3. osc.region.list

osc.region.listは、動作するOSCリージョンを設定するファイルであり、OSCシステムの起動と終了時に使用されます。リージョン名を起動する順に改行して記述します。システムの起動・終了時に、記述された順序に起動および終了します。

以下は、osc.region.listファイルにOSC00001、OSC00002の2つのOSCリージョンを設定した例です。

```
OSC00001
OSC00002
```

2.2. Tmax環境設定ファイルの設定

OSCはTmaxをベースに動作します。したがって、OSCが正常に動作するためには、Tmax環境設定ファイルにOSCが必要とするサーバーやサービスが登録されている必要があります。

OSCサーバーには、OSCリージョンに属するOSCアプリケーション・サーバーとOSCアプリケーション補助サーバー、そしてOSCユーザー・サーバー、OSCシステム・サーバーなどがあります。OSCアプリケーション・サーバーとOSCアプリケーション補助サーバーは、ユーザーのリージョン構成に従ってTmax環境設定の内容が変わり、OSCユーザー・サーバーは、ユーザーのシステム構成に従って変わります。OSCシステム・サーバーのTmax環境設定は、ユーザーのリージョン構成とは関係なく、常に同じです。

本節では、OSCシステム・サーバーのためのTmax環境設定ファイルの設定方法について説明し、次にOSCユーザー・サーバーの設定方法について説明します。

参考

リージョン構成に基づくOSCアプリケーション・サーバーと補助サーバーの設定方法については、[「3.6. Tmax環境設定ファイルの設定」](#)を参照してください。

2.2.1. サーバー・グループの設定

[SVRGROUP]セクションには、OSCシステム・サーバーの特性に応じて2つのサーバー・グループを設定します。シングル・ノード環境では各サーバー・グループが同じ設定を持ちますが、マルチノード環境では各グループの特性に応じてCOUSIN、BACKUP、LOADの設定が異なります。

サーバー・グループは、svgotpn、svgotpbの2つのグループがあります。

グループ名	説明
svgotpn	マルチノード環境でノード間の負荷分散とバックアップをサポートしないグループです。
svgotpb	マルチノード環境でノード間のバックアップはサポートしますが、負荷分散はサポートしないグループです。

以下は、シングル・ノード環境における[SVRGROUP]セクションの設定例です。

```
*SVRGROUP
svgotpn      NODENAME = "NODE1"
svgotpb      NODENAME = "NODE1"
```

以下は、NODE1、NODE2の2つのノードで構成されたマルチノード環境における[SVRGROUP]セクションの設定例です。

```
*SVRGROUP
svgotpn      NODENAME = "NODE1", COUSIN = svgotpn2, LOAD = -1
svgotpn2     NODENAME = "NODE2", LOAD = -1
```

```
svgotpb      NODENAME = "NODE1", BACKUP = svgotpb2
svgotpb2     NODENAME = "NODE2"
```

2.2.2. システム・サーバーの設定

OSCシステム・サーバーは、[SERVER]セクションに登録します。登録するサーバーは、oscmgr、oscmcsvr、oscmnsvr、oscncsvr、oscscsvr、oscdfsvr、oscjcsvrの7つのOSCシステム・サーバーです。

以下は、[SERVER]セクションの設定例です。サーバー設定はノード構成に関係なく、以下の例のように設定します。

```
*SERVER
oscmgr      SVGNAME = svgotpn, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscmcsvr    SVGNAME = svgotpn,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscmnsvr    SVGNAME = svgotpn, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscncsvr    SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscscsvr    SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscdfsvr    SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscjcsvr    SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
```

2.2.3. システム・サーバー・サービスの設定

[SERVICE]セクションには、OSCシステム・サーバーで必要とするサービスを登録します。

以下は、[SERVICE]セクションの設定例です。サービス設定はノード構成に関係なく、以下の例のように設定します。

```
*SERVICE
# oscmgr
OSCMGRSVC      SVRNAME = oscmgr
OSCMGRREGLIST  SVRNAME = oscmgr
OSCMGRTERMLIST SVRNAME = oscmgr
OSCMGRDISCONN  SVRNAME = oscmgr
OSCMGRHOURGLASS SVRNAME = oscmgr

# oscmcsvr
OSCMCSVRSMFW   SVRNAME = oscmcsvr
```

```
# oscncsvr
OSCNCSSVRDEFINE SVRNAME = oscncsvr
OSCNCSSVRDELETE SVRNAME = oscncsvr
OSCNCSSVRGET SVRNAME = oscncsvr
OSCNCSSVRQUERY SVRNAME = oscncsvr
OSCNCSSVRREWIND SVRNAME = oscncsvr
OSCNCSSVRUPDATE SVRNAME = oscncsvr
OSCNCSSVRBROWSE SVRNAME = oscncsvr

# oscscsvr
OSCSCSVRSTART SVRNAME = oscscsvr
OSCSCSVRDELAY SVRNAME = oscscsvr
OSCSCSVRCANCEL SVRNAME = oscscsvr
OSCSCSVRINFO SVRNAME = oscscsvr
OSCSCSVRCONTROL SVRNAME = oscscsvr

# oscdfsvr
OSCDFSVRBRKE SVRNAME = oscdfsvr
OSCDFSVRBRIN SVRNAME = oscdfsvr
OSCDFSVRCHCK SVRNAME = oscdfsvr
OSCDFSVRRESP SVRNAME = oscdfsvr
OSCDFSVRREIN SVRNAME = oscdfsvr
OSCDFSVRSETF SVRNAME = oscdfsvr

# oscdfsvr
OSCJCSVRFLUSH SVRNAME = oscjcsvr
OSCJCSVRWRITE SVRNAME = oscjcsvr
```

2.2.4. ユーザー・サーバーの設定

OSCユーザー・サーバーは、[SERVER]セクションに登録します。登録可能なサーバーは、OSCMQSVRです。

以下は、[SERVER]セクションの設定例です。ユーザー・サーバーはノード構成によって設定が異なることがあります。以下の例では、OSCMQ001というOSCMQSVRサーバーを設定しています。

```
*SERVER
OSCMQ001 SVGNAME = svgotpu, TARGET = oscmqsvr, MAX = 1, SVRTYPE = UCS,
          CLOPT = "-o $(SVR).out -e $(SVR).err -n"
```

2.3. TCacheの設定

マルチノード・クラスタリング機能をサポートするために、RTSDやアプリケーション・サーバー間で共有が必要な情報をデータベース・テーブルで管理しますが、更新が頻繁に発生しないテーブル

を検索する場合は、TCacheを使用することで、データベースへのアクセスによるパフォーマンスの低下を最小限に抑えられます。

TCacheの設定はノードごとに行います。使用する共有メモリ・キーとリージョン別テーブル情報を設定します。

参考

基本設定の詳細については『Tmax TCacheガイド』を参照してください。

2.4. WASサーブレットの設定

外部からの要求をOSCに送信したい場合は、OSCアプリケーションをWebアプリケーション・サーバー(WAS)にデプロイして、データを加工すると送信することができます。現在外部のクライアントがHTTPプロトコル要求をOSCに送信した場合、WASを介してOSCに転送されます。

本節では、OSCサーバーに要求を送信する際に使用されるWASのサーブレットの設定方法を説明します。

2.4.1. oschttp

oschttpは、外部からHTTP要求が送られたときに、OSCに要求を送信するためにデータを加工するサーブレットです。

OSCをインストールすると、以下のパスにサーブレット・ファイルと必要な設定ディレクトリがあります。このサーブレットを使用するには、運用環境のWASに直接デプロイしてください。

```
$OPENFRAME_HOME/osc/oschttp/
```

参考

oschttpサーブレットを使用するためにデプロイする方法については、使用されているWAS製品のマニュアルを参照してください。

oschttp.properties

oschttp.propertiesファイルに要求が送信されるOSCの情報を設定します。

```
TMAXIP = tmax_ip
TMAXPORT = tmax_port
ENDIAN = [BIG_ENDIAN | LITTLE_ENDIAN]
REGIONNAME = region_name
TCLNAME = tranclass_name
```

以下は、設定項目についての説明です。

項目	説明
TMAXIP	要求が送信されるOSCのTmaxのIPアドレスを指定します。
TMAXPORT	要求が送信されるOSCのTmaxのポート番号を指定します。
ENDIAN	OSCのオペレーティング・システムのエンディアン形式を指定します。
REGIONNAME	要求が送信されるOSCのリージョン名を指定します。
TCLNAME	要求が送信されるOSCのトランザクションクラス名を指定します。

以下は、oschttp.propertiesファイルの設定例です。

```
TMAXIP = 192.168.190.129
TMAXPORT = 8888
ENDIAN = BIG_ENDIAN
REGIONNAME = OSC00001
TCLNAME = TCL1
```


第3章 OSCリージョンの設定

本章では、OSCリージョンを構成するための設定と設定に必要な手順について説明します。

3.1. 概要

OSCリージョンは、ユーザーの業務プログラムを処理するアプリケーション・サーバーと追加的な機能を実行する補助サーバーで構成されます。補助サーバーは、TDQログ・タイプとOpenFrame Managerの[OSC]メニューを使用する機能をサポートします。

以下は、アプリケーション・サーバーを準備する手順です。

1. アプリケーション・サーバー・バイナリの作成

アプリケーション・サーバー・バイナリを作成し、サーバーの環境設定ファイルを設定します。

2. アプリケーション・サーバーDBテーブルの作成

アプリケーション・サーバーで使用するデータベース・テーブルを作成します。

3. アプリケーション・サーバー・データセットの作成

アプリケーション・サーバーで使用するデータセットを作成します。

4. アプリケーション・サーバー・リソースの登録

アプリケーション・サーバーで使用するデフォルト・リソースとユーザー・リソースを登録します。

5. Tmax環境設定ファイルの設定

Tmax環境設定ファイルにアプリケーション・サーバーとサービスを登録します。

6. アプリケーション・サーバーの起動と終了に必要なスクリプト・ファイルを準備します。

3.2. アプリケーション・サーバー・バイナリの作成

OSCアプリケーション・サーバーを使用するには、サーバー・プロセス・バイナリを作成する必要があります。**oscbuild**ツールを使用してサーバー・バイナリを作成した後、アプリケーション・サーバーの構成に従って、各サーバーに必要な環境設定ファイルを作成して設定します。環境設定の詳細については、「[第2章 OSCシステムの設定](#)」を参照してください。

3.2.1. ソース・ファイルの作成

サーバー・バイナリの作成時に必要に応じてユーザー・ソース・ファイルを作成します。OSCでは、5つの関数を使用して指定または追加することができます。

ユーザー・ソース・ファイルで下の表に示す関数のロジックを作成し、サーバーの起動および終了時に必要な追加操作を実行できます。また、非XA環境でトランザクションの開始とSYNCPOINT、SYNCPOINT ROLLBACKへの対応機能を追加することができます。

追加操作がない場合は、ソース・ファイルを作成する必要はありません。コンパイル時に、**oscbuild** ツールの-fオプションを使用してユーザー・ソース・ファイルを指定します。

OSCで各関数を呼び出す時点は以下のとおりです。

関数名	説明
otpsvrinit	非XA環境かXA環境かに関係なく、リージョン・サーバーの起動時に呼び出します。
otpsvcinit	非XA環境でトランザクションの開始時に呼び出します。
otpsvrcommit	非XA環境でユーザーやOSCによってSYNCPOINTが呼び出された際に呼び出します。
otpsvrrollback	非XA環境でユーザーやOSCによってSYNCPOINT ROLLBACKが呼び出された際に呼び出します。
otpsvrdone	非XA環境かXA環境かに関係なく、リージョン・サーバーの終了時に呼び出します。

以下は、システムが提供するOSCアプリケーション・サーバーのサンプル・ソースです。

```
int otpsvrinit(int argc, char *argv[])
{
    return 0;
}

int otpsvcinit()
{
    return 0;
}

int otpsvrcommit()
{
    return 0;
}

int otpsvrrollback()
{
    return 0;
}

int otpsvrdone()
{

```

```
    return 0;
}
```

3.2.2. コンパイルとデプロイ

oscbuildツールを使用して、アプリケーション・サーバー・バイナリを作成します。oscbuildツールは、OSCシステムがインストールされている環境に従ってコンパイル・オプションとサーバー・バイナリ名を設定できる機能を提供しています。

以下は、oscbuildを使用してLinux 64ビット環境で使用するサーバー・バイナリを作成する例です。サーバー・バイナリ名を特に指定しない場合、OSC00001が設定されます。

```
$ oscbuild -o LINUX64
```

コンパイルが完了すると、OSC00001というバイナリが上記のコマンドを実行したディレクトリに生成されます。生成されたバイナリは、名前を変更することができます。サーバー・バイナリ名は8文字以下で設定します。バイナリ・ファイルを、Tmax環境設定ファイルの該当ノードのAPPDIRオプションに設定されているディレクトリにコピーします。

参考

oscbuildツールの使用方法については『OpenFrame ツールリファレンスガイド』を参照してください。

3.3. アプリケーション・サーバーDBテーブルの作成

アプリケーション・サーバーの運用に必要なデータを保存するデータベース・テーブルを作成します。テーブルの作成には、**oscinit**ツールを使用します。

以下は、テーブルの種類と特性についての説明です。

● リージョン管理テーブル

OSCリージョンの状態を管理するための情報を保存します。

テーブル名	説明
OFM_OSC_REGION_MASTER	現在起動しているリージョンの状態情報を保存します。ノード名、起動状態、サーバー状態などを保存します。
OFM_OSC_REGION_LIST	ドメイン上で起動しているリージョンのリストを確認できます。リージョン・サーバーがダウンした場合、そのリージョン名をキーとして持つ他のシステム・テーブルのレコードが一括して削除されます。

● システム定義テーブル

OSCシステムの運用に必要なリソース定義を保存します。

テーブル名	説明
OFM_OSC_SD_GROUP	リソース・グループの定義を保存します。
OFM_OSC_SD_CONN	CONNECTIONリソースの定義を保存します。
OFM_OSC_SD_FILE	FILEリソースの定義を保存します。
OFM_OSC_SD_PROG	PROGRAMリソースの定義を保存します。
OFM_OSC_SD_TYPE_TERM	TYPETERMリソースの定義を保存します。
OFM_OSC_SD_TERM	TERMINALリソースの定義を保存します。
OFM_OSC_SD_TSMODEL	TSMODELリソースの定義を保存します。
OFM_OSC_SD_JNL_MODEL	JOURNALMODELリソースの定義を保存します。
OFM_OSC_SD_MAP_SET	MAPSETリソースの定義を保存します。
OFM_OSC_SD_WEB_SVC	WEBSERVICEリソースの定義を保存します。
OFM_OSC_SD_PIPELINE	PIPELINEリソースの定義を保存します。
OFM_OSC_SD_ENQ_MODEL	ENQMODELリソースの定義を保存します。
OFM_OSC_SD_TCPIP_SVC	TCPIPSERVICEリソースの定義を保存します。
OFM_OSC_SD_SESSION	SESSIONSリソースの定義を保存します。
OFM_OSC_SD_PARTITIONSET	PARTITIONSETリソースの定義を保存します。
OFM_OSC_SD_PROFILE	PROFILEリソースの定義を保存します。
OFM_OSC_SD_LIBRARY	LIBRARYリソースの定義を保存します。
OFM_OSC_SD_URIMAP	URIMAPリソースの定義を保存します。
OFM_OSC_SD_TRANS	TRANSACTIONリソースの定義を保存します。
OFM_OSC_SD_TRAN_CLASS	TRANCLASSリソースの定義を保存します。
OFM_OSC_SD_TDQ	TDQUEUEリソースの定義を保存します。

● RTSDテーブル

OSCサーバーの起動中に使用されるリソース情報を保存します。

テーブル名	説明
OFM_OSC_CONN	CONNECTIONリソース情報を保存します。
OFM_OSC_FILE	FILEリソース情報を保存します。
OFM_OSC_PROG	PROGRAMリソース情報を保存します。

テーブル名	説明
OFM_OSC_TERM	TERMINALリソース情報を保存します。
OFM_OSC_TSMODEL	TSMODELリソース情報を保存します。
OFM_OSC_JNL_MODEL	JOURNALMODELリソース情報を保存します。
OFM_OSC_MAP_SET	MAPSETリソース情報を保存します。
OFM_OSC_WEB_SVC	WEBSERVICEリソース情報を保存します。
OFM_OSC_PIPELINE	PIPELINEリソース情報を保存します。
OFM_OSC_ENQ_MODEL	ENQMODELリソース情報を保存します。
OFM_OSC_TCPIP_SVC	TCPIPSERVICEリソース情報を保存します。
OFM_OSC_SESSION	SESSIONSリソース情報を保存します。
OFM_OSC_PARTITIONSET	PARTITIONSETリソース情報を保存します。
OFM_OSC_PROFILE	PROFILEリソース情報を保存します。
OFM_OSC_LIBRARY	LIBRARYリソース情報を保存します。
OFM_OSC_URIMAP	URIMAPリソース情報を保存します。
OFM_OSC_TRANS	TRANSACTIONリソース情報を保存します。
OFM_OSC_TRAN_CLASS	TRANCLASSリソース情報を保存します。
OFM_OSC_TDQ	TDQUEUEリソース情報を保存します。
OFM_OSC_NETNAME	NETNAMEリソース情報を保存します。

● システム・テーブル

OSCサーバーの運用に必要な設定とシステム情報を保存します。

テーブル名	説明
OFM_OSC_GETMAIN	リージョン・プロセス間で共有されるGETMAIN領域のアドレス値を保存します。
OFM_OSC_TRAN2SVC	トランザクション名に対応するサービス名を保存します。
OFM_OSC_SVRINFO	リージョン・プロセスの状態情報を保存します。
OFM_OSC_UPDATE_FILE	FILEリソースの更新要求を保存します。
OFM_OSC_UPDATE_TDQ	TDQリソースの更新要求を保存します。
OFM_OSC_OPEN_FILE	FILEリソースの開放要求を保存します。
OFM_OSC_OPEN_TDQ	TDQリソースの開放要求を保存します。
OFM_OSC_CONFIG	OSCの一部環境設定を保存します。

テーブル名	説明
OFM_OSC_SACEE	端末IDに対応するSACEE値を保存します。
OFM_OSC_KEY2TRAN	ショートカットキーに対応するトランザクション名を保存します。
OFM_OSC_CSPG	CSPGコマンドを保存します。
OFM_OSC_TSQINFO	TSQの状態情報を保存します。
OFM_OSC_TSQDATA	TSQのデータを保存します。

• サービス・ログ・テーブル

トランザクションの統計情報を保存します。サービス・ログの詳細については「[第5章 OSCの運用](#)」を参照してください。

テーブル名	説明
OFM_OSC_OLOG	トランザクションの統計情報を保存します。

• システム・タイム・テーブル

OSCシステムのタイム・テーブル情報を保存します。

テーブル名	説明
OFM_OSC_TX_TIME	OSCシステムの時間と日付情報を保存します。

参考

1. oscinitツールの使用方法については『OpenFrame ツールリファレンスガイド』を参照してください。
 2. 各リソース定義の詳細については『OpenFrame OSCリソース定義ガイド』を参照してください。
-

3.4. アプリケーション・サーバー・データセットの作成

OSCアプリケーション・サーバーに定義されているデータセットは、サーバーを起動する前に生成されている必要があります。サーバーを起動する前にデータセットが生成されていない場合、サーバーの起動時にエラーが発生します。

データセットを作成するには、**バッチ・ジョブ**を実行するか、**IDCAMS**ユーティリティを使用します。OpenFrame/Batchがインストールされていない場合は、**idcams**ツールを使用してデータセットを作成することもできます。

本節では、**IDCAMS**ユーティリティ・スクリプトを使用してデータセットを作成する例と対応するidcamsツールの使用例を説明します。

参考

1. IDCAMSユーティリティの詳細については『OpenFrame ユーティリティリファレンスガイド』を参照してください。
 2. idcamsツールの詳細については『OpenFrame ツールリファレンスガイド』を参照してください。
 3. バッチ・ジョブを実行してデータセットを作成する方法については、『OpenFrame TJESガイド』を参照してください。
-

3.4.1. パーティション内TDQデータセットの作成

パーティション内TDQデータセットは、以下のように設定する必要があります。

- TSAMキー順データセット(KSDS)として作成します。
- データセットのキー・サイズは8に固定します。
- レコード長とレコード形式は、実際にTDQのパーティション内キューを使用する際に作成するデータ長に合わせて直接定義します。

以下は、OSC.TDQLIB.INTRA.OSC00001というパーティション内TDQデータセットを作成するIDCAMSスクリプトです。最初の行は、同じ名前のデータセットが存在する場合は削除するように指示するコマンド文です。

```
DELETE OSC.TDQLIB.INTRA.OSC00001 CLUSTER
DEFINE CLUSTER (NAME(OSC.TDQLIB.INTRA.OSC00001) -
    KILOBYTES (128,128) -
    VOLUMES (100000) -
    INDEXED KEYS (8,0) -
    RECORDSIZE (128,32760) )
```

以下は、上記のサンプル・スクリプトをユーザーが指定したUNIXファイルに保存した後、シェルでIDCAMSユーティリティを使用してTSAMデータセットを作成するスクリプトです。

```
$ IDCAMS < IDCAMS_OSC.TDQLIB.INTRA.sample
```

以下は、上記のIDCAMSユーティリティの使用例をidcamsツールで実行した例です。

```
$ idcams delete -t CL OSC.TDQLIB.INTRA.OSC00001
$ idcams define -t CL -n OSC.TDQLIB.INTRA.OSC00001 -o KS -k 8,0 -b 32768
-l 128,32760 -s 1024,128,128 -v 100000
```

3.4.2. TSQデータセットの作成

TSQデータセットは、キュー情報(Queue Information)を保存するためのデータセットとキュー・データ(Queue Data)を保存するためのデータセットを1つずつ作成します。

キュー情報TSQデータセット

キュー情報を保存するためのTSQデータセットは、以下のように設定する必要があります。

- TSAMキー順データセット(KSDS)として作成します。
- データセットのキー・サイズは16バイトです。
- レコードは64バイト以上の固定長に指定します（可変長に指定しないでください）。

以下は、OSC.TSQLIB.INFO.OSC00001というキュー情報を保存するためのデータセットを作成するIDCAMSスクリプトです。最初の行は、同じ名前のデータセットが存在する場合は削除するように指示するコマンド文です。

```
DELETE OSC.TSQLIB.INFO.OSC00001 CLUSTER
DEFINE CLUSTER (NAME(OSC.TSQLIB.INFO.OSC00001) -
    KILOBYTES (128,128) -
    VOLUMES (100000) -
    INDEXED KEYS (16,0) -
    RECORDSIZE (64,64) )
```

以下は、上記のサンプル・スクリプトをユーザーが指定したUNIXファイルに保存した後、シェルでIDCAMSユーティリティを使用してTSAMデータセットを作成するスクリプトです。

```
$ IDCAMS < IDCAMS_OSC.TSQLIB.INFO.sample
```

キュー・データTSQデータセット

キュー・データを保存するためのTSQデータセットは、以下のように設定する必要があります。

- TSAMキー順データセット(KSDS)として作成します。
- データセットのキー・サイズは18バイトです。
- レコード長とレコード形式は、実際にTSQを使用する際に、作成するデータ長に合わせて直接定義します。

以下は、OSC.TSQLIB.DATA.OSC00001というキュー・データを保存するためのデータセットを作成するIDCAMSスクリプトです。最初の行は、同じ名前のデータセットが存在する場合は削除するように指示するコマンド文です。

```
DELETE OSC.TSQLIB.DATA.OSC00001 CLUSTER
DEFINE CLUSTER (NAME(OSC.TSQLIB.DATA.OSC00001) -
    KILOBYTES (128,128) -
```



```
VOLUMES (100000) -  
INDEXED KEYS (18,0) -  
RECORDSIZE (128,32760) )
```

以下は、上記のサンプル・スクリプトをユーザーが指定したUNIXファイルに保存した後、シェルでIDCAMSユーティリティを使用してTSAMデータセットを作成するスクリプトです。

```
$ IDCAMS < IDCAMS_OSC.TSQLIB.DATA.sample
```

以下は、上記の2つのIDCAMSユーティリティの使用例をidcamsツールで実行した例です。まずキュー情報を保存するためのデータセットを作成した後、キュー・データを保存するためのデータセットを作成しています。

```
$ idcams delete -t CL OSC.TSQLIB.INFO.OSC00001  
$ idcams define -t CL -n OSC.TSQLIB.INFO.OSC00001 -o KS -k 16,0 -b 32768  
    -l 64,64 -s 1024,128,128 -v 100000  
  
$ idcams delete -t CL OSC.TSQLIB.DATA.OSC00001  
$ idcams define -t CL -n OSC.TSQLIB.DATA.OSC00001 -o KS -k 18,0 -b 32768  
    -l 128,32760 -s 1024,128,128 -v 100000
```

3.4.3. FILEデータセットの作成

FILEデータセットは、TDQデータセットやTSQデータセットとは異なり、ユーザーがFILEリソースを使用する場合に限って作成します。FILEリソースを使用するためには、システムが起動する前にFILEリソース定義のDSNAME項目に設定されているデータセットを作成する必要があります。リソース設定のOPENTIME(STARTUP)に指定されたファイルのデータセットを、ユーザーがシステムの起動前に作成していない場合、システムの起動時にエラーが発生します。

以下は、ファイル定義の例です。

```
DEFINE FILE(OSCFILE)  
    GROUP(TEST)  
    DSNAME(OSC.FILE)  
    RECORDFORMAT(V)  
    ADD(YES)  
    BROWSE(YES)  
    DELETE(YES)  
    READ(YES)  
    UPDATE(YES)
```

以下は、上記のように定義されたOSCFILEのためのデータセットを作成する例です。FILEリソース定義のDSNAMEに指定した名前を持つデータセットを定義するIDCAMS入カスクリプトを作成します。レコード長は最大1024バイトの可変長で、キーはレコードの先頭から64バイトのサイズを持つOSC.FILEを作成するスクリプトです。

```
DEFINE CLUSTER (NAME(OSC.FILE) -  
    KILOBYTES (128,128) -  
    VOLUMES (100000) -  
    INDEXED KEYS (64,0) -  
    RECORDSIZE (64,1024) ))
```

IDCAMSユーティリティを使用して、上記のサンプル・スクリプトでTSAMデータセットを作成します。

```
$ IDCAMS < IDCAMS_OSC.FILE.sample
```

以下は、上記のIDCAMSユーティリティの使用例をidcamsツールで実行した例です。

```
$ idcams define -t CL -n OSC.FILE -o KS -k 64,0 -b 32768  
-l 64,1024 -s 1024,128,128 -v 100000
```

参考

FILEリソースの詳細については『OpenFrame OSCリソース定義ガイド』を参照してください。

3.5. アプリケーション・サーバー・リソースの登録

OSCは登録必須のリソース定義が記述されている基本マクロ・ファイルを提供しています。また、OSCが提供する多様な機能を使用するために必要なリソース定義が記述されているサンプル・マクロ・ファイルも提供しています。OSCの管理者は、リージョンに基本マクロ・ファイルを登録し、使用したい機能に応じてサンプル・マクロを修正して登録してください。

3.5.1. デフォルト・リソースの登録

基本マクロ・ファイルは、アプリケーション・サーバーの動作に必要なプログラムとトランザクションを定義しています。基本マクロ・ファイル(osc.dat)は、\${OPENFRAME_HOME}/osc/resourceディレクトリに存在します。OSCのインストールによって提供されます。

基本マクロ・ファイルに定義されているリソースを登録するには、**oscsdgen**ツールを使用してOSC SDテーブルに登録します。**oscsdgen**を使用してOSCリージョンを指定してリソースを登録することもできます。

以下は、**oscsdgen**ツールを使用してOSCが提供する基本マクロ・ファイル(osc.dat)をOSC00001リージョンに登録する例です。

```
$ oscsdgen -c -r OSC00001 osc.dat
```

3.5.2. ユーザー・リソースの登録

サンプル・マクロ・ファイルには、OSCが提供する各種リソースが定義されています。サンプル・マクロ・ファイルは\${OPENFRAME_HOME}/osc/resourceディレクトリに存在します。OSCと一緒に提供されるので、OSCをインストールすれば使用できます。リソースには、CONNECTION、FILE、JOURNALMODEL、PROGRAM、TDQ、TERMINAL、TRANCLASS、TRANSACTION、TSMODEL、TYPETERMの計10種類があります。

oscsdgenツールを使用してサンプル・マクロ・ファイルをOSC SDテーブルに登録します。また、**OpenFrame Manager**の[OSC] > [System Definition]メニューからリソース定義を追加・変更・削除することができます。

参考

1. **OpenFrame Manager**の[OSC] > [System Definition]メニューについては、『OpenFrame Manager ユーザーガイド』を参照してください。
 2. OSCが提供するリソースについては『OpenFrame OSCリソース定義ガイド』を参照してください。
-

3.6. Tmax環境設定ファイルの設定

アプリケーション・サーバーとアプリケーション補助サーバー、また該当サーバーで使用するサービスをTmax環境設定ファイルに追加する必要があります。

3.6.1. アプリケーション・サーバーとサービスの登録

OSCアプリケーション・サーバーは、非対話型と対話型の2種類のサーバーがあります。1つのアプリケーション・サーバーを追加するには、両方のサーバーを登録する必要があります。アプリケーション・サーバーは、Tmax環境設定ファイルの[SERVER]セクションに登録します。

サーバーの登録情報は以下のとおりです。サーバー名以外のオプションは、Tmaxサーバーの登録方法と同じです。

サーバー	サーバー名
非対話型アプリケーション・サーバー	– TRANCLASSサーバー：リージョン名 _TRANCLASS名（例: OSC00001_TCL0） – PLTサーバー：リージョン名（例: OSC00001）
対話型アプリケーション・サーバー	– 対話型サーバー：リージョン名（例: OSC00001C）

サービスの登録情報は以下のとおりです。サービス名は、サーバー名に接尾辞を追加して登録します。TRANCLASSサーバーの場合は、サーバー名と同じサービス名を登録します。

サービス	接尾辞	サービスが実行されるサーバー
一般トランザクションを処理するサービス	なし	TRANCLASSサーバー
PLT機能処理するサービス	P	PLTサーバー
対話型トランザクションを処理するサービス	C	対話型サーバー
DPLと関数移送を処理するサービス	M	対話型サーバー

注

サービス名は対話型ではなく、**非対話型アプリケーション・サーバー名(リージョン名)に接尾辞を追加**して使用することに注意します。

各サービスの役割に応じてアプリケーション・サーバーの設定を変更して、トランザクション処理が効率的に行われるようにします。

以下は、OSC00001リージョンの非対話型サーバー(OSC00001_TCL0、OSC00001_TCL1、OSC00001)と対話型サーバー(OSC00001C)を登録し、該当サーバーにサービスを設定する例です。

```
*SERVER
OSC00001_TCL0  SVGNAME = svgbiz,
                TARGET = OSC00001,
                MIN = 1,
                MAX = 1,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err"

OSC00001_TCL1  SVGNAME = svgbiz,
                TARGET = OSC00001,
                MIN = 1,
                MAX = 1,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err"

OSC00001       SVGNAME = svgbiz,
                MIN = 1,
                MAX = 1,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err -n"

OSC00001C      SVGNAME = svgbiz,
                TARGET = OSC00001,
                CONV = 0,
                MAX = 1,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err"

*SERVICE
```

OSC00001_TCL0	SVRNAME = OSC00001_TCL0
OSC00001_TCL1	SVRNAME = OSC00001_TCL1
OSC00001P	SVRNAME = OSC00001
OSC00001	SVRNAME = OSC00001
OSC00001M	SVRNAME = OSC00001C
OSC00001C	SVRNAME = OSC00001C

3.6.2. アプリケーション補助サーバーとサービスの登録

アプリケーション補助サーバーは、アプリケーション・サーバーに追加的な機能を提供するために設定するサーバーです。

TDQログ機能を使用する場合に必要なOSC TDQログ・サーバーと、**OpenFrame Manager**の[OSC]メニューを使用するために必要なサーバーを追加することができます。

以下は、サーバー名を設定する方法です。

サーバー	サーバー名
OSC TDQログ・サーバー	リージョン名+TL（例: OSC0001TL）
OpenFrame Manager/OSCサーバー	リージョン名+OMC（例: OSC00001OMC）

以下は、アプリケーション補助サーバーを登録する例です。

```
*SERVER
OSC0001TL      SVGNAME = svgbiz,
                TARGET = osctlsvr,
                MAX = 1, SVRTYPE = UCS,
                CLOPT = "-o $(SVR).out -e $(SVR).err"

OSC00001OMC    SVGNAME = svgbiz,
                TARGET = oscossvr,
                MIN = 1,
                MAX = 5,
                SCHEDULE = RR,
                CLOPT = "-o $(SVR).out -e $(SVR).err -x OSCOSSVRSVC1:OSC00001_OMC1,
OSC00001OMC2:OSC00001_OMC2, OSCOSSVRMON:OSC00001_MON, OSCOSSVR_ST:OSC00001_ST"
```

各サーバーの実行バイナリ名は、**TARGET**オプションを使用してサーバー登録情報に設定します。各補助サーバーは、それぞれにサービスを登録する必要があります。

サーバー	サービスの指定
OSC TDQログ・サーバー	リージョン名に「_TL」を追加したサービスを登録します。
OpenFrame Manager/OSCサーバー	リージョン名に「_OMC1」、「_OMC2」、「_MON」、「_ST」を追加した3つのサービスを登録します。OMCサー

サーバー	サービスの指定
	バーは、代表サービスが要求を処理する仕組みであるため、CLOPTオプションを使用して代表サービスをサーバー登録情報に設定します。 サービス「_OMC1」と「_OMC2」は、マルチノード環境で特定のノードにサービスを要求する場合とノードに関係なくサービスを要求する場合があります。サービス「_OMC1」は、要求を処理するノードが特定されたサービスのグループであり、マルチノード環境ではROUTINGセクションを追加して、FB_NODENAMEフィールドを設定することで特定のノードにサービスを要求することができます。

以下は、アプリケーション補助サーバーのサービスを登録する例です。

```
*SERVICE
OSC00001_TL          SVRNAME = OSC00001TL
OSC00001_OMC1        SVRNAME = OSC00001OMC
OSC00001_OMC2        SVRNAME = OSC00001OMC
OSC00001_MON         SVRNAME = OSC00001OMC
OSC00001_ST          SVRNAME = OSC00001OMC
```

第4章 OSCアプリケーション

本章では、OSCアプリケーションで実際の業務ロジックが実装されたプログラムが実行される前に必要な操作を中心に説明します。

4.1. 概要

OSCサーバーで動作するアプリケーション・プログラムは、COBOL、PL/I、Cプログラミング言語で開発されます。OSC COBOL、PL/I、Cプログラムは、一般COBOL、PL/I、Cプログラムとは異なり、EXEC CICS文、EXEC DLI文、EXEC SQL文を使用してシステム・リソースにアクセスします。

アプリケーションの開発者は、作成したプログラムをコンパイルして、生成されたバイナリをデプロイします。コンパイルする前に、前処理が必要な場合は前処理を行います。デプロイされたプログラムは、OSC SDテーブルにプログラム・リソース定義を登録した後、OSCシステムを起動すると動作します。

本章では、UNIXシェルを使用した開発準備プロセスについて説明します。

参考

1. OpenFrame Studioを利用してアプリケーションを開発することもできます。OFStudioについては、『OpenFrame Studio ユーザーガイド』を参照してください。
 2. OSCアプリケーション・プログラムでマップを使用する場合は、マップの開発と準備も一緒に行われる必要があります。マップについては、『OpenFrame OSCマッピングサポートガイド』を参照してください。
-

4.2. ソースの前処理

ソースの前処理は、メインフレームCICSのために作成されたCOBOL、PL/I、Cアプリケーションのプログラム・ソースをUNIX用のCOBOL、PL/I、Cコンパイラでコンパイルできるように変換する作業です。

以下は、COBOL、PL/I、Cアプリケーションのプログラム・ソースを前処理する手順です。

● COBOL

COBOLプログラム・ソースの前処理は、以下の4段階で行われます。

1. COBOL構文の前処理(cobolprep)
2. OSC COBOL構文の前処理(osccobprep)
3. EXEC DLIコマンドの前処理(dliprep)

4. EXEC CICSコマンドの前処理(osccblpp)

EXEC CICSコマンドを使用するすべてのCOBOLプログラムは、osccblppツールにより前処理します。cobolprep、osccobprep、dliprepは、必要な場合に使用します。

- PL/I

PL/Iプログラム・ソースの前処理は、**EXEC CICS構文の前処理**プロセスで行います。EXEC CICSコマンドを使用するすべてのPL/Iプログラムは、osclippツールにより前処理する必要があります。

- C

Cプログラム・ソースの前処理は、**EXEC CICS構文の前処理**プロセスで行います。EXEC CICSコマンドを使用するすべてのCプログラムは、osccprepツールにより前処理する必要があります。

参考

本節で紹介する各ツールの詳しい使用方法については、『OpenFrame ツールリファレンスガイド』を参照してください。あるいは、システムのシェルで[-h]オプションを実行して確認することもできます。

4.2.1. cobolprep

cobolprepは、COBOL言語の構文を変換する前処理を行うツールです。

メインフレームとオープン環境は互いに異なるシステムをベースにしているため、オープン環境で提供するCOBOLコンパイラとメインフレームのコンパイラは同じではありません。そのため、オープン環境でメインフレームのCOBOLプログラムを起動するには、メインフレームで使用するCOBOL言語の構文をオープン環境に合わせて変更する前処理が必要になります。

以下の場合に、COBOLアプリケーションのプログラム・ソースの前処理が必要です。

- オープン環境用のコンパイラが認識できないか、サポートしていないCOBOL構文
- 特定の機能を使用するために、追加的にメインフレームで使用するCOBOLアプリケーションのプログラム・ソースの前処理が必要な場合

以下は、COBTEST.cobファイルを前処理して、cobolprep_COBTEST.cobファイルを現在のディレクトリに作成する例です。

```
$ cobolprep COBTEST.cob
```

4.2.2. osccobprep

osccobprepは、OSCで使用されるCOBOLプログラムの中でメインフレーム環境の特徴のためUNIXのCOBOLコンパイラによって正常に処理できない構文を前処理し、OSCシステムに合わせてコマ

ンドを追加し、デバッグなどの用途で前処理するツールです。現在は、BLL、BMSMAPBR、EDFの前処理機能があります。

前処理の結果ファイルは、結果ファイル名または接頭辞をオプションで指定しない場合、ファイル名に「osccobprep_」という接頭辞が付きます。

参考

BLL(Base Locator for Linkage)は、IBMのメインフレームCOBOLコンパイラの中で、OS/VS COBOL以下の旧バージョンでサポートする特殊な構文構造です。詳しい説明は、IBMの『CICS Application Programming Guide』を参照してください。

使用方法

以下は、osccobprepツールの実行方法です。

```
Usage: osccobprep [options] file
```

- [options]

オプション	説明
[-V]	Verboseモードで実行します。
[-a]	BMSMAPBRの前処理を行います。
[-b]	BLLの前処理を行います。
[-c]	動的呼び出し機能やCBLPSHPOP機能を使用する場合に指定します。 OSCシステムの基本機能であり、デフォルトで使用することをお勧めします。
[-d]	EDFの前処理を行います。
[-f postfix]	コピーブック・ファイルの拡張子を指定します（ピリオド(.)が使用された場合、ピリオドを含みます）。
[-p dir]	コピーブック・ファイルが存在するディレクトリ・パスを指定します。
[-o1 prefix]	前処理の結果ファイル名に付ける接頭辞を指定します。
[-o2 file]	前処理の結果ファイル名を指定します。 [-o1]オプションと[-o2]オプションが一緒に使用された場合、[-o2]オプションが優先されます。
[-h]	ツールのヘルプを表示します。
[-v]	ツールのバージョン情報を表示します。

- 入力項目

項目	説明
file	前処理するファイル名を指定します。

使用例

以下は、osccobprepツールを使用してTCOB.cobプログラムのソースコードを前処理して、osccobprep_TESTCOB.cobファイルを作成する例です。プログラムで使用するコピーブックの拡張子は「.cob」、コピーブックのディレクトリ・パスは「\$OPENFRAME_HOME/ osc/copybook」に設定しています。

```
$ osccobprep -b -f .cob -p $OPENFRAME_HOME/osc/Copybook TESTCOB.cob
```

4.2.3. osccblpp

osccblppは、OSCで使用されるCOBOLアプリケーションのプログラム・ソースに存在するEXEC CICSコマンドを前処理するツールです。特に指定しない場合は、結果ファイル名に「osccblpp_」という接頭辞が付きます。

使用方法

以下は、osccblppツールの実行方法です。

```
Usage: osccblpp [-c] [-nl] [-ne] [-n] [-V] [-p <prefix>] <file> ...  
      | osccblpp [-c] [-nl] [-ne] [-n] [-V] [-p <prefix>] [-o <output>] <file>  
      | osccblpp [-h | -v]
```

• [options]

オプション	説明
[-c]	DFHCOMMAREAを追加しません。
[-n]	EXEC CICSコマンドの前処理を行いません。
[-nl]	データ部のLINKAGEセクションと手続き部でDFHCOMMAREAとDFHEIBLKを変更しません。
[-ne]	LINKAGEセクションにDFHEIBLKを追加しません。
[-o <i>output</i>]	出力ファイルの名前を指定します。
[-p <i>prefix</i>]	出力ファイル名の接頭辞を指定します。
[-V]	Verboseモードで実行します。
[-h]	ツールのヘルプを表示します。
[-v]	ツールのバージョン情報を表示します。

• 入力項目

項目	説明
<i>file</i>	前処理するCOBOLプログラムのソース・ファイルを指定します。一度に複数のモジュールを指定することができます。

使用例

以下は、SAMPLE00.cobファイルを前処理して、osccblpp_SAMPLE00.cobファイルを作成する例です。

```
$ osccblpp SAMPLE00.cob
```

4.2.4. oscplipp

oscplippは、OSC PLIアプリケーションのプログラム・ソース内に存在するEXEC CICSコマンドを前処理するツールです。特に指定しない場合は、結果ファイル名に「oscplipp_」という接頭辞が付きます。

使用方法

以下は、oscplippツールの実行方法です。

```
Usage: oscplipp [options] file ...
```

- [options]

オプション	説明
[-d]	前処理に関連するデバッグ・メッセージを出力します。
[-f]	外部プロシージャ・コールに対し、FETCH文を追加します。
[-I]	内部スキャナーに関連するデバッグ・メッセージを表示します。
[-m]	MF COBOLと連携する場合に指定します。
[-o file]	結果ファイルの名前を指定します。
[-y]	構文解釈の段階でのデバッグ・メッセージを表示します。
[-I dir]	インクルード・ファイルを検索するディレクトリ名を指定します。
[-V]	Verboseモードで実行します。
[-h]	ツールのヘルプを表示します。
[-v]	ツールのバージョン情報を表示します。

- 入力項目

項目	説明
file	前処理するPLIプログラムのソース・ファイルを指定します。一度に複数のモジュールを指定することができます。

使用例

以下は、sample00.pliファイルを前処理して、osclipp_sample00.pliファイルを作成する例です。

```
$ osclipp sample00.pli
```

4.2.5. osccprep

osccprepは、OSCで使用するCプログラムの中でUNIXのCコンパイラによって正常に処理できない構文を前処理し、OSCシステムに合わせてコマンドを追加する前処理ツールです。

前処理の結果ファイル名や接頭辞をオプションで指定しない場合、ファイル名に「osccprep_」という接頭辞が付きます。[-o]オプションを指定していない場合、拡張子は「.c」になります。

前処理プロセスでは、OSCが提供するdfheiptr.hヘッダー・ファイルを追加し、EXEC CICSで始まるコマンドをOSCが提供する関数に変更します。

参考

[-o]オプションを指定しない場合は拡張子が「.c」に生成されるため、拡張子が「.c」のファイルは入力ファイルとして使用できません。

使用方法

以下は、osccprepツールの実行方法です。

```
Usage: osccprep [-p <prefix>] [-o <output>] [-V] <file> ...  
       | osccprep [-h | -v]
```

- [options]

オプション	説明
[-p <prefix>]	前処理の結果ファイル名に付ける接頭辞を指定します。
[-o <output>]	前処理の結果ファイルの名前を指定します。 [-p]オプションと[-o]オプションが一緒に使用された場合、[-o]オプションが優先されます。
[-V]	Verboseモードで実行します。
[-h]	ツールのヘルプを表示します。
[-v]	ツールのバージョン情報を表示します。

- 入力項目

項目	説明
file	前処理するファイル名を指定します。拡張子が「.c」のファイルは指定できません。

使用例

以下は、osccprepツールを使用してTEST.ccsプログラムのソースコードを前処理して、osccprep_TEST.cファイルを作成する例です。

```
$ osccprep TEST.ccs
```

以下は、osccprepツールを使用してTEST.ccsプログラムのソースコードを前処理して、TEST.cファイルを作成する例です。

```
$ osccprep -V -o TEST.c TEST.ccs
```

4.3. コンパイル

以下は、COBOL、PL/I、Cアプリケーションのプログラム・ソースのコンパイルに関する説明です。

• COBOL

COBOLコンパイラを利用して共有オブジェクトを作成します。共有オブジェクト名はCOBOLプログラム・ソースのPROGRAM-ID値と同じである必要があります。

• PL/I

PL/Iコンパイラを利用して共有オブジェクトを作成します。共有オブジェクト名はPL/Iプログラム・ソースのプロシージャ名と同じである必要があります。

• C

Cコンパイラを利用して共有オブジェクトを作成します。Cプログラム・ソースには共有オブジェクト名と同じ名前の関数が存在する必要があります（main関数を持つソースは、前処理の際に、main関数名をソース・ファイル名に変更します）。

参考

本節の説明で、コンパイラと共有オブジェクトの作成に関する詳細は、COBOL、PL/I、Cコンパイラのマニュアルを参照してください。

4.4. デプロイおよび適用

OSCシステムが提供するosctdlinitツールとosctdlupdateツールを使用して、コンパイルされたプログラムをOSCリージョンごとにデプロイおよび適用します。

プログラムのデプロイおよび適用方法は、以下のとおりです。

1. プログラムを保存するディレクトリを、OSCアプリケーション・サーバーの環境設定である **osc.{servername}** サブジェクト、**GENERAL** セクションの **TDLDIR** キーに設定します。TDLDIR キーに設定したディレクトリの配下に **mod** ディレクトリと **run** ディレクトリが存在しない場合は直接作成します。TDLDIR キーを設定しない場合、デフォルトで `${OPENFRAME_HOME}/osc/region/{リージョン名}/tdl` ディレクトリを参照します。
2. TDLDIR ディレクトリの配下の **config** ディレクトリに **tdl.cfg** ファイルを作成します。tdl.cfg ファイルは、インストーラーが提供するサンプル・ファイルを利用します。

以下は、tdl.cfg ファイルの作成例です。PL/I プログラムを使用する場合でも、LANG 項目は COBOL に指定します。

```
# shared memory version (1|1D|2|2D|3)
VERSION=3

# shared memory key
SHMKEY=0x3AB6

# shared memory and file creation permission
IPCPERM=0750

# number of dynamic loadable modules, rounded up to powers of 2
MAXMODULES=2000

# language environment
LANG=COBOL

# keeping extension
KEEPEXT=asmo
```

アセンブリ・モジュール(.asmo)を使用する場合は、上記例のように KEEPEXT オプションに拡張子名を指定し、**osctdlupdate** ツールを実行します。

3. OSCリージョンごとに **osctdlinit** ツールを実行します。TDLDIR ディレクトリの配下の **config** ディレクトリにある **tdl.cfg** ファイルの設定に従って初期化が行われます。
4. コンパイルされたプログラムのバイナリは、TDLDIR ディレクトリの配下の **mod** ディレクトリに保存します。
5. **osctdlupdate** ツールに OSCリージョン名とプログラム名を指定してプログラムをデプロイします。ファイルの拡張子は省略し、プログラム名のみを指定します。アセンブリ・モジュールの場合は、ファイルの拡張子を省略できません。

参考

1. **osctdlrm**ツールを使用して、使用中の共有メモリの情報を削除することができます。メモリ情報を削除した後、再度共有メモリを作成するには、**osctdlinit**ツールを使用します。
 2. 環境設定のためのサブジェクトの設定方法については『OpenFrame 環境設定ガイド』を参照してください。
-

4.5. プログラムとトランザクションの登録

管理者は、アプリケーション・プログラムを登録するためにプログラムとトランザクション定義マクロを準備し、事前にOSC SDテーブルに登録しておく必要があります。

以下は、OIVPMAINプログラムとOIVPトランザクションを定義するリソース定義マクロの例です。以下のマクロ・ファイルを**oscsdgen**ツールを使用してOSC SDテーブルに登録します。または、**OpenFrame Manager**の[OSC] > [Regions] > [System Definitions]メニューから登録・変更・削除することができます。

```
DEFINE PROGRAM(OIVPMAIN)
    GROUP(OIVP)
    LANGUAGE(COBOL)

DEFINE TRANSACTION(OIVP)
    GROUP(OIVP)
    PROGRAM(OIVPMAIN)
    TWASIZE(255)
```

参考

1. プログラムとトランザクション・リソース定義についての詳細は、『OpenFrame OSCリソース定義ガイド』を参照してください。
 2. **OpenFrame Manager**の[OSC] > [Regions] > [System Definitions]メニューについての詳細は、『OpenFrame Manager ユーザーガイド』を参照してください。
-

第5章 OSCの運用

本章では、OSCシステムの起動および終了方法と運用中のOSCシステムに関する情報を確認・変更する機能について説明します。

5.1. OSCの起動と終了

OSCシステムを起動および終了するには、`oscboot/oscdown`ツールを使用します。`oscboot/oscdown`ツールを使用して、OSCシステム全体を起動・終了したり、特定のOSCリージョンを起動・終了することができます。

参考

1. OSCシステム全体ではなく、特定サーバーだけを起動または終了するには、Tmaxが提供する`tmboot/tmdown`ツールを使用します。
 2. リージョンを起動する際には、起動前に`pfmtcacheadmin`ツールを使用してTCacheを初期化する必要があります。
-

5.1.1. OSCの起動

以下では、OSCを起動する手順を説明します。

1. OpenFrameシステム・サーバーの起動

`oscboot`ツールは、`${OPENFRAME_HOME}/config`ディレクトリにある`ofsys.seq`ファイルに設定されているサーバーを順次に起動させます。あるサーバーが起動時に他のサーバーが提供するサービスを呼び出す場合に、サービスを提供するサーバーが起動していないとサービスを呼び出したサーバーにエラーが発生します。`ofsys.seq`に設定されているサーバーを順次に起動させる機能は、このようなエラーの発生を防ぐためのものです。`ofsys.seq`ファイルは、OSCのインストール時にデフォルトで設定されています。

`oscboot`ツールは、内部的に`oscboot.pre`に記述されている作業を行った後、OpenFrameシステム・サーバーを起動させます。起動プロセスがすべて終了したら、`oscboot.post`に記述されている作業を実行します。`oscboot.pre`ファイルと`oscboot.post`ファイルは、`${OPENFRAME_HOME}/bin`ディレクトリに存在します。

2. OSCシステム・サーバーの起動

`oscboot`ツールは、エンジン内部で指定されたOSCシステム・サーバーを起動させます。

3. OSCユーザー・サーバーの起動

`oscboot`ツールは、`ofosc.seq`に記述されているOSCユーザー・サーバーを起動させます。

4. OSCリージョンの起動

`osc.region.list`に記述されているOSCリージョンを1つずつ起動させます。各リージョンのリソースを準備した後、OSCリージョンに属しているサーバーを起動させます。

以下は、各OSCリージョンに必要なリソースを準備する手順です。

- a. 共有メモリにOSCリージョンで使用するシステム・メモリ領域を作成します。
- b. 共有メモリにOSCリージョンで使用するユーザー・メモリ領域を作成します。
- c. 共有メモリにOSCリージョンで使用するTSQのための領域を作成します。
- d. タスク・コントロール機能のためのディレクトリを作成します。
- e. パーティション内TDQデータセットを初期化します。
- f. RTSDとシステム・テーブルを作成します。

リソースの準備が完了すると、OSCリージョンに属しているアプリケーション・サーバーと補助サーバーを起動させます。各OSCリージョンの起動時に、PLTPIに設定されたプログラムは、DFHPLTマクロのPROGRAM=DFHDELIM項目の設定に関係なく、順番に実行されます。PLTPIプログラムでEXEC CICS STARTコマンドにより要求されたトランザクションは、すべてのPLTPIプログラムの実行が終了した後に開始されます。PLTPIプログラムの実行がすべて終了すれば、OSCリージョンの起動が終了します。

参考

1. oscbootツールの[-r]オプションを使用して個別リージョンを起動させることができます。[-s]オプションを一緒に使用すると、リージョンで使用するリソースを作成せずに、サーバーのみを起動させることができます。
 2. DFHPLTマクロは、oscpitcツールを使用してコンパイルおよびデプロイします。oscpitcツールについての詳細は『OpenFrame ツールリファレンスガイド』を参照してください。
 3. DFHPLTマクロについての詳細は『OpenFrame OSCリソース定義ガイド』を参照してください。
-

5.1.2. OSCの終了

以下では、OSCを終了する手順を説明します。

1. OSCリージョンの終了

`oscdown`ツールは、`${OPENFRAME_HOME}/config`ディレクトリにある`osc.region.list`ファイルを参照してOSCリージョンを順次に終了します。各OSCリージョンの終了時には、まずPLTSDに設定されている前処理プログラムを実行します。OpenFrame環境設定の`osc.{servername}`サブジェクト、**GENERAL**セクションの**PSTSD**キーのVALUE項目を参照してPLTをロードし、PLTに指定されているプログラムを実行します。

PLTSDは2段階に分けられます。

- a. DFHPLTマクロのPROGRAM=DFHDELIM項目の前に設定されたプログラムが順番に実行されます。この段階で、OSCアプリケーション・サーバーは、TRANSACTIONリソース定義のSHUTDOWN(ENABLED)項目に設定されたトランザクションが要求されるか、または、OpenFrame環境設定の`osc.{servername}`サブジェクト、**GENERAL**セクションの**XLT**キーのVALUE項目に指定されたDFHXLTマクロに設定されたトランザクションが端末から要求された場合にのみ処理します。
- b. すべてのユーザー・トランザクションが終了した後、DFHPLTマクロのPROGRAM=DFHDELIM項目以降のプログラムが実行されます。OSCアプリケーション・サーバーは新規のユーザー・トランザクション要求を受け取りません。

参考

1. DFHXLTマクロは、**oscxltc**ツールを使用してコンパイルおよびデプロイします。oscxltcツールについての詳細は『OpenFrame ツールリファレンスガイド』を参照してください。
 2. DFHPLTマクロおよびDFHXLTマクロについての詳細は『OpenFrame OSCリソース定義ガイド』を参照してください。
-

上記の手順が完了すると、当該OSCリージョンに属しているアプリケーション・サーバーとアプリケーション補助サーバーを終了します。OSCリージョンが使用したリソースを削除します。

- a. 共有メモリでOSCリージョンに割り当てられたTSQの領域を削除します。
- b. 共有メモリでOSCリージョンに割り当てられたユーザー・メモリ領域を削除します。
- c. 共有メモリでOSCリージョンに割り当てられたシステム・メモリ領域を削除します。
- d. タスク・コントロールのために生成されたディレクトリを削除します。
- e. RTSDとシステム・テーブルを削除します。

2. OSCユーザー・サーバーの終了

oscdownツールは、`${OPENFRAME_HOME}/config`ディレクトリの**ofosc.seq**に記述されているすべてのサーバーを、起動するときと逆の順序で終了させます。

3. OSCシステム・サーバーの終了

oscdownツールは、エンジン内部で指定されたOSCシステム・サーバーを、**oscboot**ツールで起動時と逆の順序で終了します。

4. OpenFrameシステム・サーバーの終了

oscdownツールは、`${OPENFRAME_HOME}/config`ディレクトリに**ofsys.seq**ファイルが存在する場合、起動時と逆の順序ですべてのサーバーを終了させます。この機能は、あるサーバーが終了時に他のサーバーが提供するサービスを呼び出した場合に、サービスを提供するサーバーが先に終了してサービスを呼び出すサーバーが終了できない現象を防ぎます。

上記の手順がすべて完了した後、oscdownツールは、**oscdown.pre**に記述された作業を先に実行した後、システム・サーバーを終了させます。システム・サーバーを終了した後は、**oscdown.post**に記述された作業を実行します。oscdown.preファイルとoscdown.postファイルは、`$(OPENFRAME_HOME)/bin`ディレクトリに存在します。

参考

oscdownツールの[-i]オプションを使用してシステムを終了する場合は、OSCリージョンのソースが削除されないため、必要な場合にのみ使用してください。また、OSCリージョンを再起動するときは、oscbbootツールの[-m]オプションを指定して再起動します。

5.2. OSCの運用

OSCの管理項目は大きく、システム・レベルの管理項目とリージョン・レベルの管理項目に分けられます。OSCシステム・レベルの管理項目には、システム・ログ検索機能、システム・リソースの検索および管理機能があります。リージョン・レベルの管理項目には、リージョン情報の検索、OSC SDテーブル管理、RTSD管理などの一般項目と、トランザクション、端末、マップなどのリージョンのリソース項目があります。

管理者はシステム全体に影響を与える管理項目とリージョンに影響を与える管理項目を区別して運用する必要があります。

5.2.1. システムの運用

OSCはシステム・サーバーとリージョンの運用のために、OSCシステム情報の検索、ログ検索、リソース検索機能を提供します。OSCが提供するシステム・レベルの管理項目は以下のとおりです。

- RESP、RESP2コードの検索機能
- システム・サーバーとアプリケーション・サーバーのログ検索機能
- システム・リソースの検索および管理機能：名前付きカウンター、スケジュールされたトランザクション、端末

上記のシステム・レベルの運用は、**OpenFrame Manager**の[OSC]メニューから行えます。

RESPコードの検索は、**oscresp**ツールによっても実行できます。端末接続情報の検索および管理は、**oscadmin**ツールによっても実行できます。

参考

OpenFrame Managerを利用したOSCシステムの運用方法については、『OpenFrame Manager ユーザーガイド』を参照してください。

5.2.2. リージョンの運用

OSCは、リージョン情報の検索、OSC SDおよびRTSDの管理、EDFなどの機能を提供します。また、リージョンで使用する各リソースの詳細管理機能も提供します。

OSCが提供するリージョン・レベルの管理項目は以下のとおりです。

● リージョンの一般情報の検索

リージョンの一般的な情報を検索する機能を提供します。リージョンの基本情報（APPLID、JOBID、SYSIDなど）とシステム・メモリの使用状態、スプールの使用状態、サーバー・ログなどを検索することができます。

この機能は、**OpenFrame Manager**の[OSC] > [Regions]メニューで使用できます。

● リージョンのプロセス情報の検索

起動しているリージョンのプロセス情報を検索できます。管理者は、サーバー・プロセスのPID、Tmaxが管理するSPRI、トランザクション関連情報、そしてユーザー・アプリケーションがEDF機能を使用してデバック中であるかどうかなどをリアルタイムで確認することができます。また、特定のサーバー・プロセスがユーザー・アプリケーションのエラーやシステム・エラー、またはその他の問題のため作業を実行できない状況にある場合は、そのプロセスを終了して実行中の作業を中断させ、新しいプロセスを起動させます。ただし、プロセスの終了機能は使用しているリソースの整合性を保証しないため、注意して使用する必要があります。

この機能は、**OpenFrame Manager**の[OSC] > [Regions]メニューで使用できます。

● OSC SDテーブルの管理

OSC SDテーブルをより便利に管理できるように、リソース定義の登録・検索・変更・削除機能を提供します。**oscsdgen**ツールを使用してOSC SDテーブルにリソース定義を登録したり、定義されているリソースを変更および削除することができます。また、**oscsddump**ツールを使用して定義されているリソースを検索することができます。

この機能は、**OpenFrame Manager**の[OSC] > [Regions] > [System Definitions]メニューで使用できます。

● OSC RTSDテーブルの管理

次の3つの方法で、運用中のOSCシステムをリアルタイムで管理することができます。

– 方法1)

OpenFrame Managerの[OSC] > [Regions] > [Runtime Resources]メニューからデータベース・テーブルにロードされたリソース定義を検索・変更・削除することができます。ただし、リソースを削除すると、運用しているアプリケーション・サーバーでそのリソースを使用できなくなります。

– 方法2)

OSCが提供するRTSDツールを使用して、データベース・テーブルにロードされたリソース定義を検索・登録・変更することができます。RTSDツールは、リソース定義の削除機能は提供しません。RTSDツールには、現在運用中のアプリケーション・サーバーのRTSD情報を更新する

ためのoscrtsdupdateツールと、RTSD情報をマクロ形式のテキスト・ファイルとして作成するためのoscrtsddumpツールがあります。

– 方法3)

システム・プログラミング・インターフェースを使用して管理用のOSCアプリケーションを作成します。

注

ランタイム・リソースの場合は、変更可能なリソースが限られているので、リストを確認した後に変更してください。

リージョンで使用するリソースの詳細管理項目は以下のとおりです。

項目	説明
トランザクション	トランザクションが実行されるリージョン、トランザクションを要求した端末、トランザクションに対応するプログラム名、トランザクションの実行開始時間など、トランザクションに関する詳細情報を検索する機能を提供します。 トランザクション・リソースの詳細検索機能は、OpenFrame Managerの[OSC] > [Regions] > [Transaction Status]メニューで使用できます。
ストレージ	GETMAINコマンドを使って、使用中のストレージ領域を詳細検索する機能を提供します。割り当てられたストレージ領域に関する情報やGETMAINを使用しているプロセスの情報を確認することができます。 ストレージ領域に関する検索機能は、OpenFrame Managerの[OSC] > [Regions] > [Storage]メニューで使用できます。
端末	アプリケーションに接続している端末の一覧とネット名の検索、ゲートウェイ番号の検索機能などを提供します。アプリケーションに接続している端末の情報をリアルタイムで管理できます。 端末の詳細検索機能は、OpenFrame Managerの[OSC] > [Terminals]メニューで使用できます。
TDQ/TSQ	アプリケーション・サーバーで使用しているTDQ/TSQの一覧の検索機能やTSQレコードの検索機能を提供します。また、TDQのレコードをTSQに移動する機能や、TSQのレコードをTDQにコピーする機能も提供します。これらの機能により、TDQとTSQをより便利に管理することができます。 TDQとTSQの管理機能は、OpenFrame Managerの[OSC] > [Regions] > [Queues]メニューで使用できます。

参考

OpenFrame Managerの[OSC]メニューの詳しい使用方法については、『OpenFrame Manager ユーザーガイド』を参照してください。

5.2.3. ログ・レベルの管理

OSCシステムでは、OSCシステム・サーバーおよびユーザー・サーバーのログ・レベルとリージョンのログ・レベルを別々に管理します。

OSCシステム・サーバーおよびユーザー・サーバーのログ・レベルは、OpenFrame環境設定の**osc** サブジェクト、**GENERAL**セクションの**SYSTEM_LOGLVL**キー値で管理します。各サーバーは起動時にこの設定値を読み込んで適用します。各リージョンのログ・レベルは、**oscadmin**ツールを使用して確認および変更することができます。変更された値は、OSCシステム・サーバーのログ・レベルとは異なり、リアルタイムで該当リージョンに反映されます。

ログ・レベルは3つに分けられます。以下は、各ログ・レベルについての説明です。

レベル	説明
E(Error)レベル	エラーが発生した場合のログ情報だけが記録されます。
I(Information)レベル	Eレベルより詳しいログ情報が記録されます。問題が発生したときに、Eレベルより詳細な情報が収集できますが、ログ・ファイルのサイズは大きくなる可能性があります（デフォルト値）。
D(Debug)レベル	問題が発生したときに、エンジンでどの操作が行われていたか、エンジンのどの部分で問題が発生したかを把握できる詳細なログ情報が記録されます。ログ・ファイルのサイズは大きくなります。

参考

1. oscサブジェクトの設定については『OpenFrame 環境設定ガイド』を参照してください。
 2. oscadminツールについては『OpenFrame ツールリファレンスガイド』を参照してください。
-

以下は、OSC00001リージョンのログ・レベルを確認する例です。

```
oscadmin -l OSC00001
```

以下は、OSC00001リージョンのログ・レベルをDebugに設定する例です。

```
oscadmin -l OSC00001:D
```

5.3. セキュリティ管理

OSCシステムのセキュリティ管理はTACF(Tmax Access Control Facility)をベースにしています。TACFは、OpenFrameシステムにアクセスするユーザーを認証するプロセスにより、許可されていないユーザーからシステムを保護し、システムのリソースに対する権限を持たないユーザーの不正アクセスを防ぐ機能を提供します。

TACFではリージョン・サーバーごとにセキュリティ管理を設定することができます。これは、OpenFrame環境設定の`osc.{servername}`サブジェクト、**SAF**セクションで設定できます。管理者は、セキュリティの範囲をリソース全体にするか、個別リソースに制限するかを選択することができます。

参考

リージョン・サーバーのセキュリティ設定についての詳細は、『OpenFrame 環境設定ガイド』を参照してください。

5.3.1. ユーザー・セキュリティ

OSCリージョンは、TACFにユーザーを登録することで、認可されていないユーザーからのアクセスを防ぐ機能を提供します。ユーザーはサインオンによりアクセス権限を得たトランザクションやリソースにアクセスします。

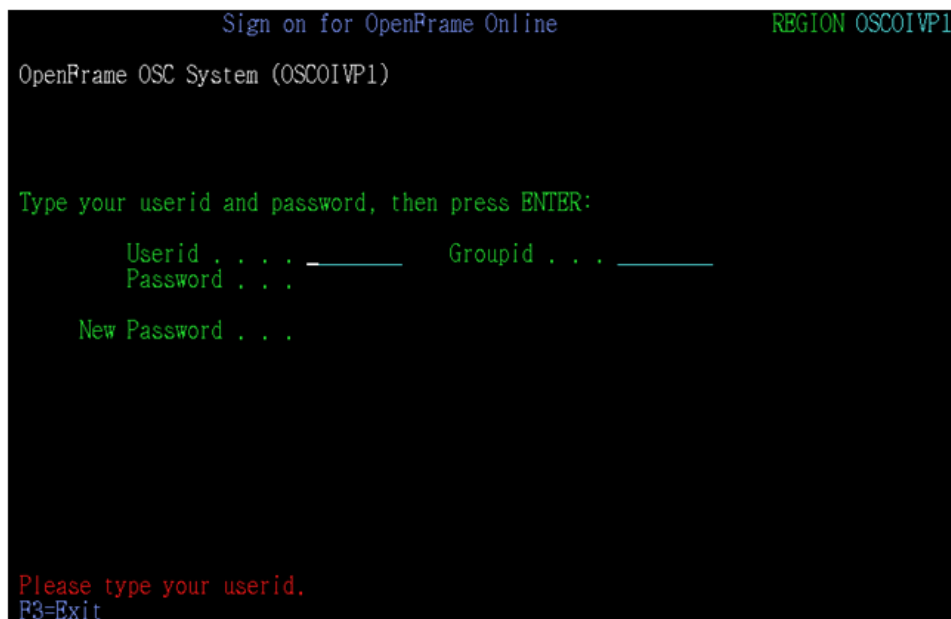
以下は、TACFを利用してOSCアプリケーションのデフォルト・ユーザーであるCICSUSERをCICSGRPグループに登録する例です。

```
ADDGROUP CICSGRP
ADDUSER CICSUSER DFLTGRP(CICSGRP) NOPASSWORD
```

ユーザーは端末に接続してCESNによりサインオンし、トランザクションやリソースにアクセスすることができます。OSCシステムを使用した後は、CESFにより安全にサインオフして、不正アクセスを防ぎます。CESNとCESFを利用せずに、アプリケーション・プログラムでSIGNON、SIGNOFFコマンドを使用して直接認証することも可能です。ユーザーが別途の認証を経していない場合、デフォルト・ユーザー権限でアプリケーション・サーバーを動作することになります。

以下は、OSCシステムでユーザーのサインオンをサポートするデフォルトのトランザクションCESNの画面です。

[図 5.1] デフォルト・トランザクションCESN画面



参考

1. ユーザーの登録方法の詳細については『OpenFrame TACF管理者ガイド』を参照してください。
 2. ユーザーの認証処理コマンドについては『OpenFrame OSC開発者ガイド』を参照してください。
-

5.3.2. リソース・セキュリティ

OSCリージョンは、ユーザーのリソース要求に対するセキュリティ機能を提供します。個別リソースに対する要求がある場合、当該ユーザーにリソースに対するアクセス権限があるかどうかを把握して、アクセスの可否を決めます。

OSCシステムでセキュリティ・サービスを提供する個別リソースは、以下のとおりです。

- トランザクション
- TDQ
- ファイル
- 開始されたトランザクション
- プログラム
- TSQ

以下は、個別リソースのセキュリティのためにTACFが使用するデフォルト・リソース・クラスの名前です。リソース単位でセキュリティを適用するには、デフォルト・メンバー・リソース・クラス

を使用し、リソース・グループ単位でセキュリティを適用するには、デフォルト・グループ・リソース・クラスを使用します。

デフォルト・クラス	対象	区分
TCICSTRN	トランザクション	メンバー
GCICSTRN	トランザクション・グループ	グループ
DCICSDCT	TDQ	メンバー
ECICSDCT	TDQグループ	グループ
FCICSFCT	ファイル	メンバー
HCICSFCT	ファイル・グループ	グループ
ACICSPCT	開始されたトランザクション	メンバー
BCICSPCT	開始されたトランザクション・グループ	グループ
MCICSPPT	プログラム	メンバー
NCICSPCT	プログラム・グループ	グループ
SCICSTST	TSQ	メンバー
UCICSTST	TSQグループ	グループ

個別リソースのセキュリティ機能を使用するには、以下の手順に従います。

1. セキュリティ・ポリシーを適用するリソースをTACFのデフォルト・メンバー・リソース・クラスに登録します。あるいは、そのリソースがメンバーとして登録されているリソース・グループ・プロファイルをデフォルト・グループ・クラスに登録します。
2. リソースのアクセス一覧にアクセスするユーザーとユーザーの権限を登録します。
3. OpenFrame環境設定の**osc.{servername}**サブジェクト、**SAF**セクションの**SEC**キーと各リソースのセキュリティ・キーのVALUE項目を「YES」に設定します。

以下は、OIVPトランザクションをTCICSTRNとGCICSTRNに登録し、CICSUSERにREAD権限を与える例です。トランザクション単位でセキュリティ・サービスを適用するにはTCICSTRNを利用し、トランザクション・グループ単位でセキュリティ・サービスを適用するにはGCICSTRNを利用します。

```
RDEFINE TCICSTRN OIVP UACC(NONE)
PERMIT OIVP CLASS(TCICSTRN) ID(CICSUSER) ACCESS(READ)
RDEFINE GCICSTRN OIVPGRP UACC(NONE) ADDMEM(OIVP, OIBR, ...)
PERMIT OIVPGRP CLASS(GCICSTRN) ID(CICSUSER) ACCESS(READ)
```

以下は、OIVPFILEをFCICSFCTとHCICSFCTに登録し、CICSUSERにUPDATE権限を与える例です。リソース単位でセキュリティ・サービスを適用するにはFCICSFCTを利用し、リソース・グループ単位でセキュリティ・サービスを適用するにはHCICSFCTを利用します。

```
RDEFINE FCICSFCT OIVPFILE UACC(NONE)
PERMIT OIVPFILE CLASS(FCICSFCT) ID(CICSUSER) ACCESS(UPDATE)
RDEFINE HCICSFCT OIVPGRP UACC(NONE) ADDMEM(OIVPFILE)
PERMIT OIVPGRP CLASS(HCICSFCT) ID(CICSUSER) ACCESS(UPDATE)
```

OSCシステムは、TDQ、TSQ、プログラム、開始されたトランザクションに対しても同じ手順でセキュリティ・サービスを提供します。

参考

1. ユーザーの登録方法の詳細については『OpenFrame TACF管理者ガイド』を参照してください。
 2. 個別リソースの詳細については『OpenFrame OSCリソース定義ガイド』を参照してください。
-

5.4. ログ情報

ログの種類は、Tmaxの内部プロセスが作成するログ、OSCアプリケーション・サーバーまたはシステム・サーバーが作成するログ、OSCアプリケーション・サーバーで実行されたサービス情報を保存するログ、OSCシステム・コマンドの操作ログなどに分けられます。

5.4.1. Tmaxログ

Tmaxログは、Tmax環境設定ファイルに記述されたノード・セクションのSLOGDIR項目、TLOGDIR項目に設定されたディレクトリに生成されます。SLOGDIRには、Tmaxシステム・メッセージを保存するファイルが生成されます。TLOGDIRには、XAトランザクションが発生したときに、トランザクションに関するメッセージを保存するファイルが生成されます。

5.4.2. サーバー・ログ

OSCアプリケーション・サーバーとシステム・サーバーが作成するログには、**OUTログ**と**ERRログ**があります。この2つのログは、Tmax環境設定ファイルに記述されたサーバーのノード・セクションの**ULOGDIR**項目に設定されたディレクトリに保存されます。ULOGDIRは自由に設定できますが、`$(OPENFRAME_HOME)/log`配下のディレクトリに設定することをお勧めします。

OUTログ

OSCアプリケーション・サーバーとシステム・サーバーがstdoutに記録するすべてのログは、CLOPTセクションに**[-o]**オプションを使用して指定したファイルに出力されます。OSCシステム・サーバーでは、OpenFrame環境設定の**osc**サブジェクト、**GENERAL**セクションの**SYSTEM_LOGLVL**キー値に設定されたログ・レベルに従ってログが記録されます。OSCアプリケーション・サーバーと補助サーバーでは、該当OSCリージョンのログ・レベルに従ってログが記録されます。

● フォーマット

以下は、OUTログのフォーマットです。

時間 ログ・レベル メッセージ・コード [10:モジュール名:SPRI:PID] ログ・メッセージ

項目	説明
時間	日付を除いたHHMMSSタイプのタイムスタンプです。
ログ・レベル	E、I、D、Tのうち1つが設定されます。 – E(Error): システムでエラーが発生した場合に出力されるメッセージで、ユーザーが正しくない情報を入力したり、システム内部でエラーが発生した場合に出力されます。 – I(Information): システムがユーザーに情報を与えるためのメッセージです。サーバーの各種情報が出力されます。 – D(Debug): システム・デバッグ用に出力されるメッセージです。 – T(Test): 入出力データがある場合に、詳細デバッグ・ログが出力されます。
メッセージ・コード	ログ・メッセージの固有のコード番号です。
[10:モジュール名:SPRI:PID]	– 10: OSCの製品番号です。 – モジュール名: メッセージを出力するOSCシステムのモジュール名です。 – SPRI: アプリケーション・サーバーのTmax SPRI値です。 – PID: アプリケーション・サーバーのプロセスIDです。
ログ・メッセージ	ログ・メッセージの内容

● 例

以下は、OSC00001アプリケーション・サーバーのOUTログの例です。

```
172703 I OSC0011I [10:CICSRES:59:19582] OSC00001 server boots
- resource manager initialization completed
172703 I OSC1705I [10:DFHSIPLT:57:19584] PLTPI : PLT suffix(I1)
173905 E OSC2010E [10:CICS:57:21240] : cics command error - EIBRESP =
0x00000001b(PGMIDERR),
EIBRESP2 = 0x000000001, EIBFN = 0x0e0e(HANDLE ABEND), VALFLAG = 0x18000001,
FLAG = 0x20000000
```

参考

oscサブジェクトの設定方法については『OpenFrame 環境設定ガイド』を参照してください。

ERROログ

OSCアプリケーションまたは接続されている他のOpenFrame製品モジュールがstderrに記録するすべてのログは、CLOPTセクションに[-e]オプションを使用して指定したファイルに出力されます。

5.4.3. サービス・ログ

トランザクション・サーバーは、トランザクションの終了時にログ・テーブル(OFM_OSC_OLOG)にログを保存します。osclogrptツール、またはOpenFrame Managerを利用してトランザクションの統計情報を確認できます。

ログ・テーブルで管理する項目は以下のとおりです。

項目	説明
リージョン名	サービスを処理するサーバーの名前です。
トランザクション日付	トランザクションが発生した日付です。
トランザクション時刻	トランザクションが発生した時刻です。
ノード名	トランザクションを実行したノード名です。
SYSID	OSCシステムのSYSIDです。
トランザクション・クラス名	トランザクションを実行したトランザクション・クラス名です。
SPRI	トランザクションを実行したサーバーのTmax SPRIです。
PID	トランザクションを実行したプロセスのIDです。
トランザクションID	トランザクションの名前です。
トランザクション時間	トランザクションの実行にかかった時間です。
トランザクションCPU使用時間	トランザクションの実行にかかったCPU時間です。
ユーザーID	トランザクションを要求した端末にログインしたユーザーのIDです。
端末IP	トランザクションを要求した端末のIPアドレスです。
端末ネット名	端末のネット名です。
システム・エラーコード	システムのベース・エンジンであるTmaxから返された処理結果値です。
ユーザー・リターンコード	トランザクションを処理したユーザー・プログラムが返した処理結果値です。

5.4.4. 操作ログ

操作ログは、\${OPENFRAME_HOME}/log/cmdディレクトリのosc_cmd_{YYYYMMDD}.logファイルに保存されます。OSCの起動と終了、TDLメモリの変更など、OSCシステム・コマンドの実行結果を記録します。

- フォーマット

以下は、操作ログのフォーマットです。ログ・レベルは、OSCシステム・コマンドが成功した場合はS(Success)、失敗した場合はE(Error)になります。ログ・レベル以外の項目は、[サーバー・ログのOUTログ](#)と同じです。

時間 ログ・レベル モジュール名 ログ・メッセージ			
項目	説明		
ログ・レベル	SとEのいずれかが設定されます。 – S(Success): OSCシステム・コマンドの処理が成功した場合です。 – E(Error): OSCシステム・コマンドの処理が失敗した場合です。		

- 例

以下は、操作ログの例です。

```
135017 S OSCB00T OSC system servers boot start
135017 S OSCB00T OSC system servers boot successfully
135017 S OSCB00T OSC region(OSC00001) boot start
135029 S OSCB00T OSC region(OSC00001) PLTPI ok
135029 S OSCB00T OSC region(OSC00001) boot ok
135035 S OSCTDLUPDATE OSC region(OSC00001) module(PGM00001) tdlupdate ok
135041 S OSCDOWN OSC region(OSC00001) shutdown start
135101 S OSCDOWN OSC region(OSC00001) PLTSD(suffix:) ok
135102 S OSCDOWN OSC region(OSC00001) shutdown ok
135102 S OSCDOWN OSC system servers shutdown start
135102 S OSCDOWN OSC system servers shutdown successfully
```

5.4.5. 監視ログ

監視ログは、OSC監視サーバーが動作するアカウントのSMFデータセットに自動で生成されます。ユーザー・トランザクションが実行されたときに、トランザクションに関連するOSCシステム情報とリソース情報をログとして記録します。ただし、監視ログを記録するためには、SMF機能を使用するための環境設定やデータセットの作成を事前に行う必要があります。SMF機能が設定されていない場合、監視ログは生成されず、誤作動する恐れがあります。

SMFデータセットに保存されるログのフォーマットは、OSC監視サーバーの環境設定によって異なります。

参考

SMFの詳細については、IBMガイドの『CICS Customization Guide』のCICSモニタリング関連項目を参照してください。

5.5. サーバーの復旧

OSCシステム・サーバー・プロセスとアプリケーション・サーバー・プロセスは、内部的なエラーやユーザー・プログラムの問題のため、オペレーティング・システムにより強制終了される場合があります。またハードウェアの問題により、OSCシステムを起動しているノード全体が予想外の終了につながることもあります。また何らかの問題で、アプリケーション・サーバー・プロセスが内部的にリンクされている他の製品や他のOSCサーバーと接続が切れることもあります。

5.5.1. OSCシステム・サーバーの復旧

OSCシステム・サーバーでサーバー・プロセスに問題が発生すると、Tmax環境設定の**RESTART**オプションにより新しいサーバー・プロセスが起動されます。内部で使用するサーバー情報構造体に問題が発生しない限り、新しいプロセスは以前の状態を復旧し、OSCアプリケーション・サーバーでサービスを継続して提供します。ただし、OSCDFSVRは、再起動されたときに既存の接続情報がすべて初期化されます。

マルチノード環境では、システム・サーバーのOSCNCSVR、OSCSCSVR、OSCDFSVRが動作していたノードで障害が発生すると、新しいサーバー・プロセスを他のノードで再起動するように設定します。

各システム・サーバーは、障害が発生したときに以下のような特徴を持ちます。

サーバー	障害時の動作
OSCNCSVR	他のノードで再起動された場合でも、OpenFrame環境設定の osc サブジェクト、 GENERAL セクションのNCS_FILEキーに共有ディスクの同一ファイルが指定されている場合は、継続してサービスを提供できます。
OSCSCSVR	バックアップ設定を行った場合は残っている要求を処理し、バックアップ設定を行っていない場合は新規要求だけを処理します。
OSCDFSVR	実行していた既存のEDF機能はすべてエラー処理されます。EDF機能を再開する必要があります。

参考

oscサブジェクトの設定方法については『OpenFrame 環境設定ガイド』を参照してください。

5.5.2. OSCアプリケーション・サーバーと補助サーバーの復旧

OSCアプリケーション・サーバーでサーバー・プロセスが予想外のエラーによって終了された場合、OSCアプリケーション・サーバーはアプリケーション・レベルのリカバリ管理を行います。問題が発生したトランザクションは異常終了され、復旧可能なリソースはロールバックされます。その後、Tmax環境設定のRESTARTオプションにより自動または手動で新しいサーバー・プロセスが起動され、次のトランザクションを処理します。ノードが異常終了した場合は、Tmaxエンジンがリカバリ管理を行います。

OSCアプリケーション・サーバー・プロセスは起動時に、TSAMデータセットを使用するためにTiberoサーバーまたは他のデータベース製品と接続しますが、ネットワーク・エラーなどの予期せぬ問題のため、接続が切断されることがあります。ファイル、補助TSQ、パーティション内TDQなど、TSAMにアクセスしようとするトランザクション内部の要求に対してアプリケーションにエラー応答を返し、該当トランザクションの復旧可能なTSAMデータセットのアクセス要求はロールバックされます。以降、同じサーバー・プロセスから要求されたトランザクションは、新しい接続が確立されるまで待機状態に置かれます。接続タイムアウト期間内に再接続が確立されない場合、トランザクションはTSAMへの接続時にもう一度エラーを発生させ、終了します。

TACF製品を使用するためにも、データベース製品に接続する必要があります。この場合もTSAMへの接続と同様に、ネットワーク・エラーなどの予期せぬ問題のため接続が切断されることがあります。そのような場合、TACFへの要求はエラーを発生させ、ユーザー・アプリケーションに関連エラーを返し、次のTACF要求がある際に再接続を試みて接続を回復します。

OSCアプリケーション・サーバーは補助サーバーのOSC TDQログ・サーバーとTCP/IPソケット通信でデータを送受信しますが、ネットワーク・エラーなどの問題が発生した場合は、該当ログTDQへのアクセスは失敗し、ユーザー・アプリケーションの次のアクセス要求時に再接続を試みて接続を回復します。

参考

リソースの使用に関する詳細は、『OpenFrame OSC開発者ガイド』を参照してください。

5.6. 日時の設定

OSCシステム・サーバーは基本的にCICSのASKTIMEコマンドで現在起動しているシステム・サーバーの日付と時間を取得してOSCシステムの日付として設定します。しかし、必要によっては特定の時間にOSCシステム全体または特定のユーザー・プログラムをテストしなければならない状況が発生することがあります。そのためOSCでは、OSCシステム全体または特定のリージョンやユーザー・プログラムの日付と時間をユーザーが任意に設定できるTX_TIMEという機能を提供します。

参考

TX_TIMEの設定については、『OpenFrame Manager ユーザーガイド』の「第4章 OSC」を参照してください。

5.6.1. 日時設定の準備

以下では、TX_TIME機能を使用するために必要な準備手順を説明します。

1. DBテーブルの確認

システム・テーブルのOFM_OSC_TX_TIMEを確認します。（OSCのインストール時に、oscinitツールにより生成されます。）

2. OSCの設定

OpenFrame環境設定のoscサブジェクト、**GENERAL**セクションの**ENABLE_TX_TIME**キー値を「YES」に指定します。

参考

oscサブジェクトの設定方法については『OpenFrame 環境設定ガイド』を参照してください。

3. TCacheの設定

pfmtcache.cfgファイルに以下の内容を設定します。（pfmtcache.cfgファイルは、OSCのインストール時にデフォルトで含まれています。）

```
# cache for OFM_OSC_TX_TIME
CACHE_NAME=OFM_OSC_TX_TIME
SIZE_MEM=32           # the total cache memory size in kilo-bytes
SIZE_HASH=32          # the number of hash key (MAX=65536)
SIZE_KEY=14           # the number of digits of the index column
SIZE_REC=38           # the size of a single record in bytes
INV_TIMEOUT=10        # invalidation timeout in sec
```

5.6.2. 日時の設定

TX_TIME機能は、DBテーブルに保存されたデータを使用してOSCシステムの日付と時間を設定します。

以下は、TX_TIME DBテーブル項目についての説明です。

項目	説明
CONTROL_TYPE	ユーザーの日付と時間を設定するKEY_IDのタイプです。 – Transaction (TX) : KEY_IDのタイプがトランザクションの場合 – Terminal (TM) : KEY_IDのタイプが端末の場合 – User (UR) : KEY_IDのタイプがユーザーの場合 – Region (RG) : KEY_IDのタイプがリージョンの場合

項目	説明
	各タイプの優先順位は、Transaction > Terminal > User > Regionの順になります。
KEY_ID	ユーザーの日付と時間を設定するターゲットのIDを指定します。ID範囲の設定をサポートする「*」を最大1つまで含めることができます。
USER_DATE	日付を「YYYYMMDD」の形式で指定します。
TIME_TYPE	指定する時間のタイプです。 – Absolute (A) : ユーザーが設定した時間に固定したい場合 – Minus (M) : システム時間を基準にユーザーが設定した時間を引きたい場合 – Plus (P) : システム時間を基準にユーザーが設定した時間を足したい場合
USER_TIME	時間を「hhmmss」の形式で指定します。
LAST_REG_USER	最後にデータを保存したユーザーの名前です。
LAST_REG_TIME	ユーザーが最後にデータを保存した時間です。「YYYYMMDD hhmmss」の形式で指定します。
LAST_UPDT_USER	最後にデータを修正したユーザーの名前です。
LAST_UPDT_TIME	ユーザーが最後にデータを修正した時間です。「YYYYMMDD hhmmss」の形式で指定します。

5.6.3. 日時の変更

以下では、TX_TIME DBテーブルのデータの日付と時間を変更する方法について説明します。

1. TCacheの初期化

TCacheに保存されているデータを初期化して、新しい日付と時間をロードする準備をします。データの初期化は、TCacheの管理者ツールのpfmtcacheadminを使用するか、OpenFrame Managerを利用して行います。

以下は、pfmtcacheadminツールを使用してTCacheにロードされたTX_TIME DBテーブルのデータを初期化する例です。

```
pfmtcacheadmin -i OFM_OSC_TX_TIME
```

2. 日時の更新

OpenFrame ManagerのTX_TIME機能を利用して、TX_TIME DBテーブルのデータの日付と時間を更新します。

参考

32ビット環境では2038年問題のため、2038年1月19日が過ぎると、1970年1月1日にリセットされるため、2038年以降の時間は使用しないようにしてください。

5.7. リソースの自動インストール

OSCシステム・サーバーを運用するには、サーバーの運用に必要なリソースをシステム定義に定義する必要がありますが、一部リソースはシステム定義に直接定義しなくても、OSCシステムがリソースを自動でインストールするようにすることができます。自動でインストールするリソース・モデルを事前にシステム定義に定義しておく、リソースの自動インストール(AutoInstall)機能により、システム定義に定義されていないリソースの使用要求があったときに、あらかじめ定義したモデルを基準にリソースを作成します。

5.7.1. プログラムの自動インストール

以下は、プログラムの自動インストールを準備する手順です。

1. 環境設定

OpenFrame環境設定の`osc.{servername}`サブジェクト、**AUTINST**セクションの**PGAEXIT**キー値として自動インストール制御プログラム名を指定します。

参考

`osc.{servername}`サブジェクトの設定方法については、『OpenFrame 環境設定ガイド』を参照してください。

2. 制御プログラムの作成

OSC環境設定に定義された制御プログラムは、ユーザーがシステム定義に定義されていないプログラムを実行したときに、OSCシステム内部で呼び出されます。

制御プログラムは、インストールするプログラム名を入力として受けて、ユーザーが作成したロジックに従ってプログラム・リソース・モデル名を返します。結果として、OSCはリソース・モデルと同じ属性を持つプログラム・リソースを新規作成して実行させます。

以下は、プログラム名('TARGETPGM')を入力として受けて、プログラム・リソース・モデル名('MODELPGM')を返す制御プログラムの作成例です。

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. OSCPGACC.  
ENVIRONMENT DIVISION.  
DATA DIVISION.  
WORKING-STORAGE SECTION.
```

```

* Commarea constants
01 PGAC-CONSTANTS.

* Module type
03 PGAC-TYPE-PROGRAM      PIC X VALUE '1'.
03 PGAC-TYPE-MAPSET       PIC X VALUE '2'.
03 PGAC-TYPE-PARTITIONSET PIC X VALUE '3'.

* Language
03 PGAC-ASSEMBLER        PIC X VALUE '1'.
03 PGAC-COBOL            PIC X VALUE '2'.
03 PGAC-PLI              PIC X VALUE '3'.
03 PGAC-C370             PIC X VALUE '4'.
03 PGAC-LE370            PIC X VALUE '5'.

* CEDF status
03 PGAC-CEDF-YES         PIC X VALUE '1'.
03 PGAC-CEDF-NO          PIC X VALUE '2'.

* Data location
03 PGAC-LOCATION-BELOW     PIC X VALUE '1'.
03 PGAC-LOCATION-ANY       PIC X VALUE '2'.

* Execution key
03 PGAC-CICS-KEY         PIC X VALUE '1'.
03 PGAC-USER-KEY         PIC X VALUE '2'.

* Load attribute
03 PGAC-RELOAD           PIC X VALUE '1'.
03 PGAC-RESIDENT         PIC X VALUE '2'.
03 PGAC-TRANSIENT        PIC X VALUE '3'.
03 PGAC-REUSABLE         PIC X VALUE '4'.

* Share status
03 PGAC-LPA-YES          PIC X VALUE '1'.
03 PGAC-LPA-NO           PIC X VALUE '2'.

* Execution set
03 PGAC-DPLSUBSET        PIC X VALUE '1'.
03 PGAC-FULLAPI          PIC X VALUE '2'.

* Dynamic status
03 PGAC-DYNAMIC-YES      PIC X VALUE '1'.
03 PGAC-DYNAMIC-NO       PIC X VALUE '2'.

* Concurrency
03 PGAC-QUASIRENT        PIC X VALUE '1'.
03 PGAC-THREADSAFE       PIC X VALUE '2'.
03 PGAC-REQUIRED         PIC X VALUE '3'.

* Api
03 PGAC-CICSAPI          PIC X VALUE '1'.
03 PGAC-OPENAPI          PIC X VALUE '2'.

* JVM
03 PGAC-JVM-YES          PIC X VALUE '1'.
03 PGAC-JVM-NO           PIC X VALUE '2'.

* Return code
03 PGAC-RETURN-OK        PIC X VALUE '1'.

```

```

03 PGAC-RETURN-DONT-DEFINE-PROG PIC X VALUE '2'.

LINKAGE SECTION.
* Include Program Autoinstall commarea
01 DFHCOMMAREA.
    COPY DFHPGACO.

PROCEDURE DIVISION.
    IF PGAC-MODULE-TYPE = PGAC-TYPE-PROGRAM
        IF PGAC-PROGRAM = 'TARGETPGM'
            MOVE 'MODELPGM' TO PGAC-MODEL-NAME
        END-IF
    END-IF.
    MOVE PGAC-RETURN-OK TO PGAC-RETURN-CODE.
    EXEC CICS RETURN END-EXEC.

```

3. プログラム・モデルの定義

システム定義に自動インストールするプログラム・モデルを定義します。一般的なプログラム・リソース定義と同じ方法で定義します。プログラムが自動インストールされる際に、ここで定義したモデルの属性を取得します。

付録 A. Tmax環境設定ファイルの例

以下は、Tmax環境設定ファイルの例です。OSCシステム・サーバー、OSCアプリケーション・サーバーと補助サーバー、TCacheに関連する項目の設定が記述されています。DOMAINやNODEなどのセクションは、OpenFrame/BaseのTmax環境設定ファイルを参照してください。

```
*SVRGROUP
svgotpu      NODENAME = "NODE1"
svgotpn      NODENAME = "NODE1"
svgotpb      NODENAME = "NODE1"
svgbiz       NODENAME = "NODE1"

*SERVER
oscmgr       SVGNAME = svgotpn, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscmcsvr     SVGNAME = svgotpn,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscmnsvr     SVGNAME = svgotpn, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"

oscncsvr     SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscscsvr     SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscdfsvr     SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"
oscjcsvr     SVGNAME = svgotpb, MAX = 1, SVRTYPE = UCS,
             CLOPT = "-o $(SVR).out -e $(SVR).err"

OSC00001     SVGNAME = svgbiz,
             MIN = 1,
             SCHEDULE = FA,
             CLOPT = "-o $(SVR).out -e $(SVR).err -n"

OSC00001C    SVGNAME = svgbiz,
             TARGET = OSC00001,
             CONV = 0,
             MIN = 4,
             MAX = 4,
             SCHEDULE = FA,
             CLOPT = "-o $(SVR).out -e $(SVR).err -n"

OSC000010MC  SVGNAME = svgbiz,
```

```

        TARGET = oscossvr,
        MIN = 1,
        MAX = 5,
        SCHEDULE = FA,
        CLOPT = "-o $(SVR).out -e $(SVR).err -x OSCOSSVRSVC1:OSC00001_OMC1,
OSCOSSVRSVC2:OSC00001_OMC2, OSCOSSVRMON:OSC00001_MON, OSCOSSVR_ST:OSC00001_ST"

OSC00001TL      SVGNAME = svgbiz, MAX = 1, SVRTYPE = UCS,
                TARGET = osctlsvr,
                CLOPT = "-o $(SVR).out -e $(SVR).err"

OSC00001_TCL0   SVGNAME = svgbiz,
                TARGET = OSC00001,
                MIN = 1,
                MAX = 2,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err -n"

OSC00001_TCL1   SVGNAME = svgbiz,
                TARGET = OSC00001,
                MIN = 1, "
                MAX = 2,
                SCHEDULE = FA,
                CLOPT = "-o $(SVR).out -e $(SVR).err -n"

*SERVICE
OSCMGRSVR      SVRNAME = oscmgr
OSCMGRREGLIST  SVRNAME = oscmgr
OSCMGRTERMLIST SVRNAME = oscmgr
OSCMGRDISCONN  SVRNAME = oscmgr

OSCMCSVRSMFW   SVRNAME = oscmcsvr

OSCSCSVRSTART  SVRNAME = oscscsvr
OSCSCSVRDELAY  SVRNAME = oscscsvr
OSCSCSVRCANCEL SVRNAME = oscscsvr
OSCSCSVRINFO   SVRNAME = oscscsvr
OSCSCSVRCONTROL SVRNAME = oscscsvr

OSCNCSVRDEFINE SVRNAME = oscncsvr
OSCNCSVRDELETE SVRNAME = oscncsvr
OSCNCSVRGET    SVRNAME = oscncsvr
OSCNCSVRQUERY  SVRNAME = oscncsvr
OSCNCSVRREWIND SVRNAME = oscncsvr
OSCNCSVRUPDATE SVRNAME = oscncsvr
OSCNCSVRBROWSE SVRNAME = oscncsvr

```



```

OSCDFSVRBRKE      SVRNAME = oscdfsvr
OSCDFSVRBRIN      SVRNAME = oscdfsvr
OSCDFSVRCHCK      SVRNAME = oscdfsvr
OSCDFSVRRESP      SVRNAME = oscdfsvr
OSCDFSVRREIN      SVRNAME = oscdfsvr
OSCDFSVRSETF      SVRNAME = oscdfsvr

OSCJCSVRFLUSH     SVRNAME = oscjcsvr
OSCJCSVRWRITE     SVRNAME = oscjcsvr

OSC00001P         SVRNAME = OSC00001
OSC00001M         SVRNAME = OSC00001C
OSC00001_TL       SVGNAME = OSC00001TL
OSC00001_OMC1     SVRNAME = OSC00001OMC
OSC00001_OMC2     SVRNAME = OSC00001OMC
OSC00001_MON      SVRNAME = OSC00001OMC
OSC00001_ST       SVRNAME = OSC00001OMC
OSC00001          SVRNAME = OSC00001
OSC00001C         SVRNAME = OSC00001C

OSC00001_TCL0     SVRNAME = OSC00001_TCL0
OSC00001_TCL1     SVRNAME = OSC00001_TCL1

#####
# Tcache configuration sample
#####
*SVRGROUP
OPFMGRP01         NODENAME = "NODE1"

*SERVER
TPFMAGENT         SVGNAME = OPFMGRP01, MIN = 1, MAX = 1,
                  CLOPT = "-o $(SVR).log -e $(SVR).log"

*SERVICE
SPFMAGENT         SVRNAME = TPFMAGENT

```


付録 B. TCache設定ファイルの例

以下は、TCache設定ファイルの例です。

```
# the configuration file of TCACHE
SHMKEY=0x70005      # the key of shared memory
SHMKEY=0x70005      # the key of shared memory
IPCPERM=0600        # permission of the shared memory
SIZE_LOCAL=65535     # L1 cache size in kilo-bytes
LIB_PTHREAD=libpthread.so  # the pthread library file name
INVALIDATE_TYPE=0    # multi-node invalidate type.
AGENT_SVC=SPFMAGENT # multi-node sync. svc

#base configurations
CACHE_NAME=ofconfig # subject
SIZE_MEM=65535      # total table size in KB: about 2KB for each entries, and there
                    # are about 400 entries in OpenFrame
SIZE_HASH=32        # the number of hash key (MAX=65536)
SIZE_KEY=324        # subject(64) + section(128) + key(128): subject>>section>>key
                    # (324)
SIZE_REC=2372       # key(324) + value(2048)
INV_TIMEOUT=1

#cache for OSC00001
CACHE_NAME=OSC00001 # the name of cache
SIZE_MEM=32767      # the total cache memory size in kilo-bytes
SIZE_HASH=32        # the number of hash key (MAX=65536)
SIZE_KEY=30         # the number of digits of the index column
SIZE_REC=2078       # the size of a single record in bytes
INV_TIMEOUT=1       # invalidation timeout in sec

# cache for REGION MASTER
CACHE_NAME=OFM_OSC_REGION_MASTER # the name of cache
SIZE_MEM=32         # the total cache memory size in kilo-bytes
SIZE_HASH=32        # the number of hash key (MAX=65536)
SIZE_KEY=8          # the number of digits of the index column
SIZE_REC=40         # the size of a single record in bytes
INV_TIMEOUT=1       # invalidation timeout in sec

#cache for OFM_OSC_TX_TIME
CACHE_NAME=OFM_OSC_TX_TIME      # the name of cache
SIZE_MEM=32                     # the total cache memory size in kilo-bytes
SIZE_HASH=32                    # the number of hash key (MAX=65536)
```

SIZE_KEY=14	# the number of digits of the index column
SIZE_REC=38	# the size of a single record in bytes
INV_TIMEOUT=1	# invalidation timeout in sec

索引

C

cobolprep, 34

D

Debugログ・レベル, 49

DL/I controlマネージャー, 6

E

Errorログ・レベル, 49

ERRログ, 55

EXCIクライアント, 7

EXEC CICS, 33

EXEC DLI, 33

EXEC SQL, 33

F

File Controlマネージャー, 6

FILEデータセット, 27

I

IDCAMS, 24

idcams, 24

Interval Controlマネージャー, 6

J

Journalマネージャー, 6

M

Mapping Supportマネージャー, 7

N

NCSマネージャー, 6

O

ofosc.seq, 11, 12

ofsys.seq, 11, 12

openframe_osc.conf, 11

OSC RTSDテーブルの管理, 47

OSC SDテーブル, 5

OSC SDテーブルの管理, 47

osc.region.list, 11, 12

oscadmin, 46, 49

oscbboot, 43, 45

oscbuild, 21

osccblpp, 36

osccobprep, 34

osccprep, 38

OSCDFSVR, 4

OSCDFSVRサーバー障害, 57

oscdwn, 44, 45

oschttp, 16

oschttp.properties, 16

OSCJCSVR, 5

OSCMCSVR, 4

OSCMGR, 4

OSCMNSVR, 5

OSCMQSVR, 5

OSCNCSVR, 4

OSCNCSVRサーバー障害, 57

OSCOSSVR, 6

osclipp, 37

oscrep, 46

OSCSCSVR, 4

OSCSCSVRサーバー障害, 57

oscsdgen, 28

osctdlinit, 39

osctdlupdate, 39

OSCTLSVR, 6

OSCアプリケーション・サーバー, 29

OSCシステム・サーバー, 4

OSCDFSVR, 4

OSCJCSVR, 5

OSCMCSVR, 4

OSCMGR, 4

OSCMNSVR, 5

OSCNCSVR, 4

OSCSCSVR, 4

OSCツール

- osccblpp, 36
- osccobprep, 34
- osccprep, 38
- osclipp, 37
- OSCユーザー・サーバー, 5
 - OSCMQSVR, 5
- OSCリージョン, 5
- OSCリージョンの補助サーバー, 5
 - OSCOSSVR, 6
 - OSCTLSVR, 6
- OSC環境設定
 - cobnet, 11
 - osc, 11
 - osc.{oscmqsvrname}, 12
 - osc.{osctlsvrname}, 11
 - osc.{servername}, 11
- OUTログ, 53

P

- Program Controlマネージャー, 6

R

- RTSD, 5
- RTSDテーブル, 22

S

- SERVERセクション, 14, 15
- SERVICEセクション, 14
- Spoolマネージャー, 7
- Storageマネージャー, 6
- SVRGROUPセクション, 13

T

- TACF, 50
- TACFマネージャー, 6
- Task Controlマネージャー, 6
- TCacheの設定, 15
- TDQ/TSQ, 48
- TDQマネージャー, 6
- Terminalマネージャー, 7
- tadmin, 3
- Tmaxログ, 53

- Tmax環境設定ファイルの設定
 - SERVERセクション, 14, 15
 - SERVICEセクション, 14
 - SVRGROUPセクション, 13
- TN3270/TN3270Eエミュレーター, 7
- TSQデータセット, 26
- TSQマネージャー, 6

W

- WASサブレットの設定, 16

あ

- アプリケーション補助サーバー, 31

か

- 監視ログ, 56

さ

- Infoログ・レベル, 49
- 操作ログ, 56
- サーバーの復旧, 57
- サーバー・ログ, 53
 - ERRログ, 55
 - OUTログ, 53
- サービス・ログ, 55
- サービス・ログ・テーブル, 24
- システムの運用, 46
- システム・タイム・テーブル, 24
- システム・テーブル, 23
- システム定義テーブル, 22
- ストレージ, 48
- ソースの前処理, 33

た

- 端末, 48
- トランザクション, 48

な

- 日時の設定, 58

は

パーティション内TDQデータセット, 25

ら

リソースの自動インストール, 61

リソース・マネージャー, 6, 7

リージョンのログ・レベル, 49

 D(Debug)レベル, 49

 E(Error)レベル, 49

 I(Information)レベル, 49

リージョンの運用, 47

リージョン管理テーブル, 21

ログ, 53

